REPUBLIQUE DU SENEGAL

Un Peuple - Un But - Une Foi

Orange Digital Center

Memoire de fin de cycle - Specialite

Developpeur Full Stack Web et Mobile

KYA (Keur Ya Aicha)

Sujet

Conception et realisation d'une

plateforme de gestion locative multi-roles

pour la gestion des clients, locations,

paiements, depots, documents et notifications

Presente par

Papa Malick Teuw

+221 77 171 90 13

malickteuw.devweb@gmail.com

Nature du projet

Projet personnel de fin d'etudes

Duree: 21 jours

Thies, Senegal

Encadre par

M. Birane Bailla Wone

Coach developpeur / Encadrant academique

Annee academique 2025-2026

Dossier de soutenance complet incluant analyse, UML,

architecture, implementation, deploiement et annexes code.

Fiche signaletique du projet

Champ	Valeur
Titre du projet	Conception et realisation d'une plateforme de gestion locative multi-roles pour la gestion des clients, locations, paiements, depots, documents et notifications
Nom de la solution	KYA (Keur Ya Aicha)
Auteur	Papa Malick Teuw
Telephone	+221 77 171 90 13

Champ	Valeur
Email	malickteuw.devweb@gmail.com
Etablissement	Orange Digital Center
Specialite	Developpeur Full Stack Web et Mobile
Nature	Projet personnel de fin d'etudes
Encadrant academique	M. Birane Bailla Wone - Coach developpeur / Encadrant academique
Zone geographique	Thies, Senegal
Duree	21 jours
Lien frontend	https://keur-ya-aicha-frontend.vercel.app
Lien backend API	https://bakend-next-saas-gestion-client.onrender.com/api
Documentation API	https://bakend-next-saas-gestion-client.onrender.com/docs

> *Ce projet vise a fournir une solution complete de gestion locative multi-roles permettant au Super Admin, a l administrateur et aux utilisateurs du service de collaborer autour d un noyau unique: clients, locations, paiements, depots, documents, imports et notifications.*

Dedicace

Je dedie ce travail a ma famille, a toutes les personnes qui m ont

soutenu pendant mon parcours, ainsi qu a tous ceux qui croient

qu une solution numerique bien pensee peut simplifier la gestion

du quotidien.

Remerciements

J adresse mes sincères remerciements a l Orange Digital Center

pour le cadre d apprentissage, a mon encadrant academique M.

Birane Bailla Wone pour ses orientations, ainsi qu a toutes les

personnes qui ont contribue de pres ou de loin a la realisation de

ce projet.

Resume

Le projet KYA (Keur Ya Aicha) est une plateforme de gestion locative multi-roles realisee pour digitaliser les processus de gestion des clients, locations, paiements, depots, documents et imports de donnees. Il repond au besoin d une administration centralisee, traçable et moderne, capable de fonctionner aussi bien sur le web que sur une application desktop Electron.

La solution distingue clairement les responsabilites du Super Admin, qui pilote la gouvernance, l approbation des comptes et la supervision, et celles de l administrateur, qui gere ses propres clients, ses locations, ses paiements et ses documents. Une attention particuliere a ete accordee a la securite, a l audit, aux permissions fines, a la maintenance de plateforme, au blocage automatique des IP suspectes et a la seconde authentification du Super Admin.

L architecture retenue repose sur un frontend React/Vite/TypeScript avec Zustand et composants reutilisables, ainsi qu un backend Next.js structure selon les principes POO et SOLID, avec Prisma et PostgreSQL. Le projet inclut en plus un module d import Excel/CSV/JSON avec mapping, previsualisation editable, journalisation des erreurs et pages de retour, un module d abonnement administrateur avec controle des regles metier, et un service de notification transactionnelle Brevo base sur des evenements et

des ecouteurs.

Le present dossier rassemble l ensemble des livrables necessaires a une soutenance complete: contexte, problematique, objectifs, UML, architecture, implementation detaillee, extraits de code exacts, liens de deploiement, documentation technique et annexes.

Abstract

KYA (Keur Ya Aicha) is a multi-role rental management platform designed to digitalize the administration of clients, rentals, monthly payments, deposits, documents, imports and transactional notifications. The project targets a practical, auditable and modern workflow for administrators and Super Admin users.

The solution combines a React/Vite/TypeScript frontend with a layered Next.js backend built around object-oriented and SOLID principles. Core features include role-based access, admin approval, configurable governance, platform maintenance, import runs with editable preview and validation, subscription payments, PDF receipts, audit logs and Brevo email notifications driven by events and listeners.

Glossaire

Sigle	Definition
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
JWT	JSON Web Token

RBAC	Role-Based Access Control
CSRF	Cross-Site Request Forgery
TOTP	Time-based One Time Password
UML	Unified Modeling Language
SPA	Single Page Application
ORM	Object Relational Mapper
CI/CD	Continuous Integration / Continuous Deployment
KYA	Keur Ya Aicha

Sommaire

1. Introduction generale
2. Chapitre 1 - Contexte et problematique
3. Chapitre 2 - Analyse fonctionnelle et UML
4. Chapitre 3 - Methodologie et organisation
5. Chapitre 4 - Architecture technique frontend / backend
6. Chapitre 5 - Modelisation des donnees et securite
7. Chapitre 6 - Implementation detaillee des grandes fonctionnalites
8. Chapitre 7 - Deploiement, tests et exploitation
9. Chapitre 8 - Discussion, limites et perspectives
10. Conclusion generale
11. Annexes techniques

Table des figures

Reference	Intitule	Ancre
Figure 1	Contexte general du projet	#contexte-general
Figure 2	Vue globale existante du projet	#vue-globale-projet
Figure 3	Use case global de l'application	#use-case-global
Figure 4	Use case dedie a l'import des clients	#use-case-import
Figure 5	Activite d'approbation d'un administrateur	#activite-approval
Figure 6	Activite d'import de clients	#activite-import
Figure 7	Activite de paiement abonnement administrateur	#activite-payment
Figure 8	Sequence d'approbation administrateur	#sequence-approval
Figure 9	Sequence login et seconde authentification	#sequence-login
Figure 10	Sequence d'import clients	#sequence-import
Figure 11	Sequence paiement abonnement administrateur	#sequence-payment
Figure 12	Architecture globale de la solution	#arch-global
Figure 13	Architecture frontend	#arch-frontend
Figure 14	Architecture backend	#arch-backend
Figure 15	Architecture de deploiement	#arch-deploy
Figure 16	Modele de donnees principal	#model-data

Reference	Intitule	Ancre
Figure 17	Diagramme de classes techniques	#class-diagram

Table des tableaux

Reference	Intitule
Tableau 1	Stack technique retenue
Tableau 2	Acteurs et responsabilites
Tableau 3	Phasage du projet sur 21 jours
Tableau 4	Cartographie modules frontend / backend
Tableau 5	Liens de deploiement et documentation
Tableau 6	Routes frontend principales
Tableau 7	Endpoints backend disponibles

Introduction generale

Le secteur locatif s appuie encore tres souvent sur des fichiers Excel disperses, des echanges WhatsApp non historises et des documents stockes sans gouvernance commune. Cette organisation produit des doublons, des oublis de paiement, des difficultes de suivi et une faible traçabilite.

Dans ce contexte, le projet KYA a ete concu comme une reponse concrete a un besoin de centralisation. La plateforme devait permettre a un Super Admin de gouverner l ensemble du service, de controler l acces des administrateurs, de piloter les politiques de

maintenance et de suivre les journaux d audit. Elle devait aussi permettre a chaque administrateur de gerer uniquement son portefeuille de clients, ses locations, ses paiements, ses documents et ses imports.

L objectif du projet ne se limitait pas a produire des ecrans. Il s agissait surtout de mettre en place une architecture fiable, securisee et evolutive, capable de supporter une version web, une version desktop Electron, une API backend industrialisable, un modele de donnees relationnel cohérent, et des integrations externes comme Cloudinary pour les fichiers et Brevo pour les notifications.

Ce document retrace donc a la fois la vision fonctionnelle et la realite technique du projet. Il presente le contexte, les choix d architecture, les diagrammes UML, la mise en oeuvre des principales fonctionnalites, les preuves d implementation dans le code, les liens de deploiement ainsi que les perspectives d evolution.

Chapitre 1 - Contexte et problematique

1.1 Presentation de l'etablissement et du cadre

Le projet a ete realise dans le cadre du parcours Developpeur Full Stack Web et Mobile a l Orange Digital Center. L environnement de formation privilegie une approche par projet, oriente la realisation de produits concrets et met l accent sur l autonomie, la qualite de code, le travail par iteration et la capacite a livrer une solution deployable.

1.2 Presentation du projet KYA

KYA signifie Keur Ya Aicha. La plateforme couvre la gestion locative complete: comptes,

entreprises, clients, locations, paiements mensuels, cautions, documents, imports de donnees et notifications. Le produit est concu comme un SaaS de gestion locative avec gouvernance centralisee. Son architecture permet un usage web sur Vercel et un usage desktop via Electron.

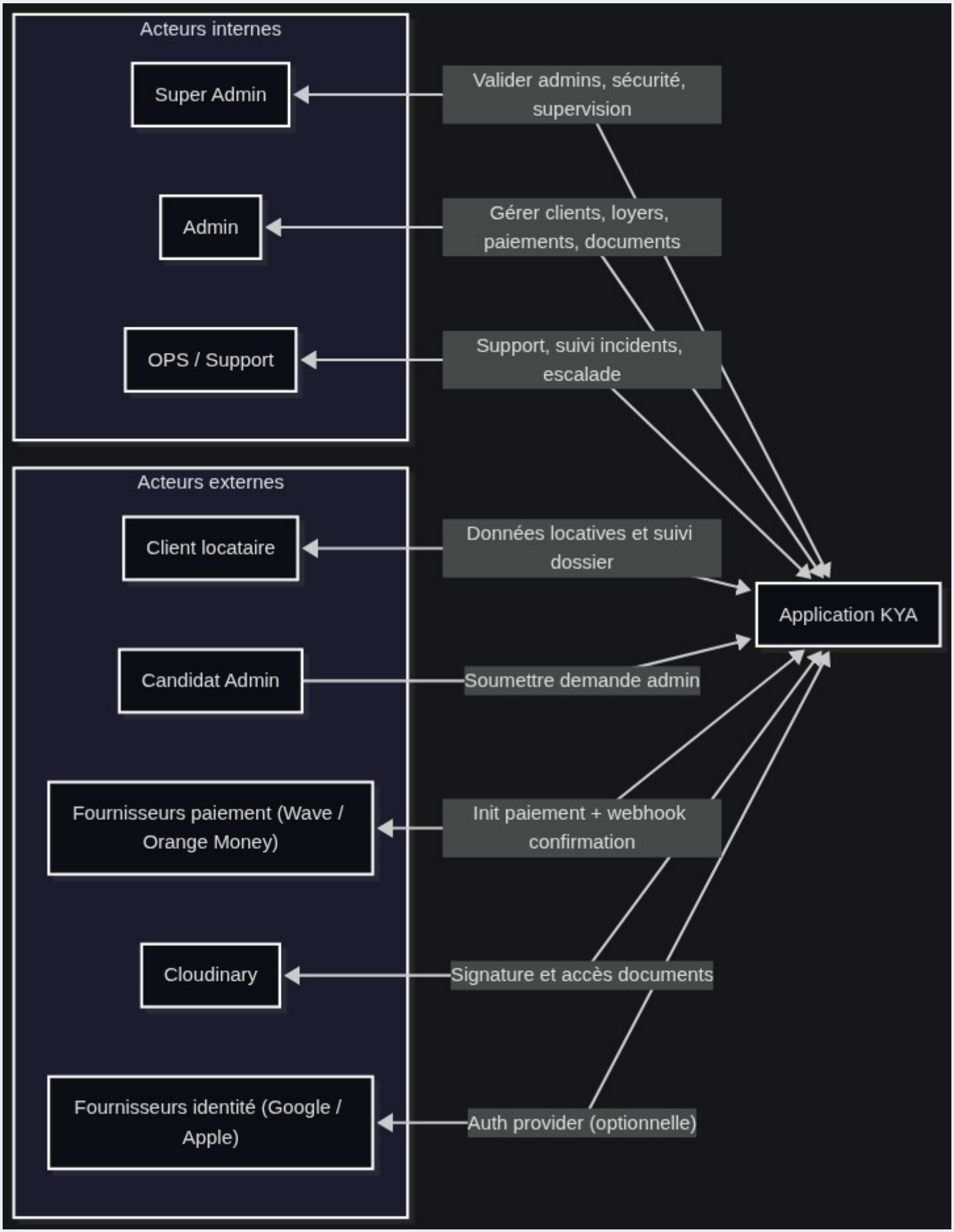


Figure 1. Contexte general du projet KYA

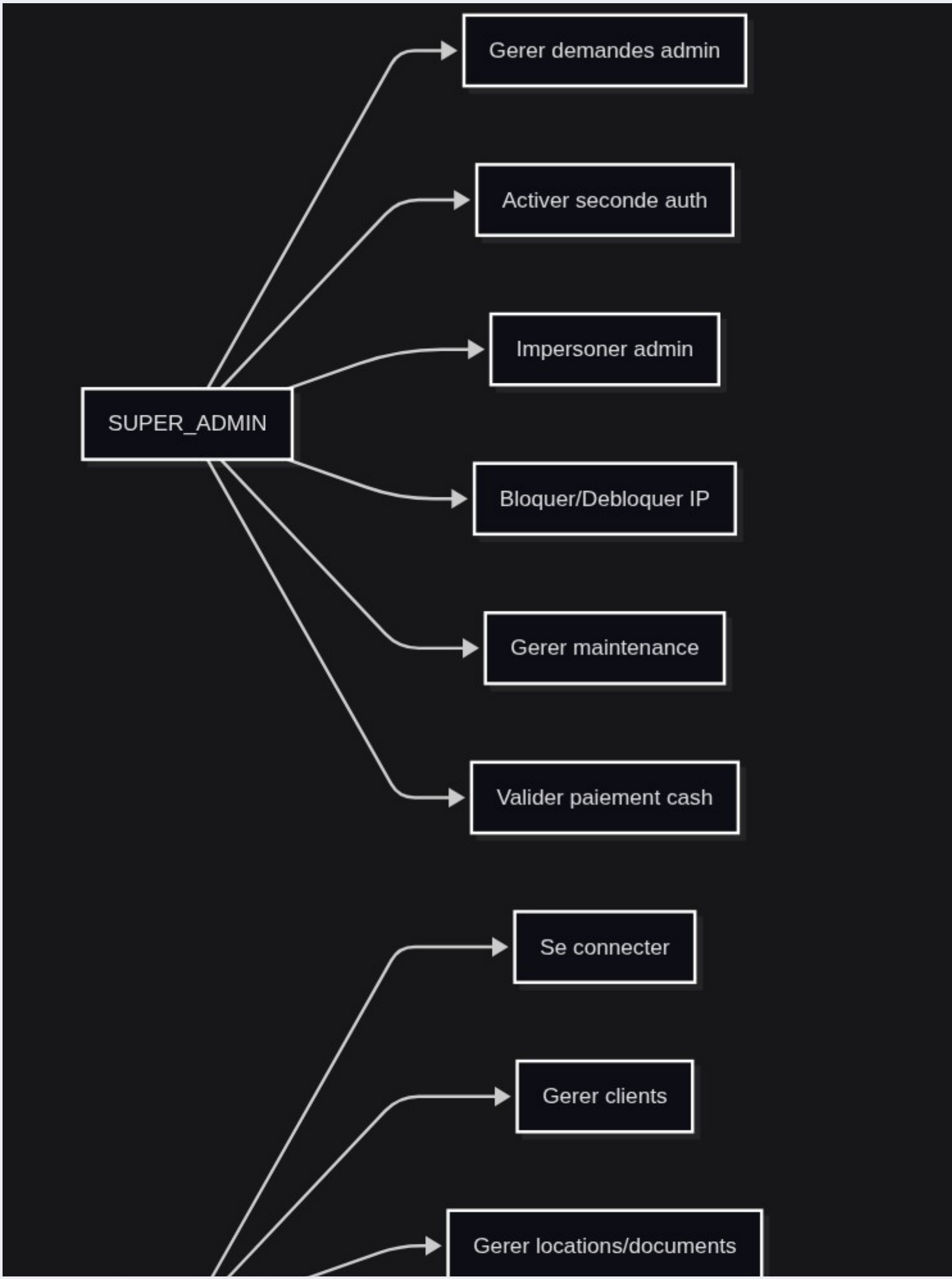

SUPER_ADMIN
Gerer demandes admin
Activer seconde auth
Impersoner admin
Bloquer/Debloquer IP
Gerer maintenance
Valider paiement cash
Se connecter
Gerer clients
Gerer locations/documents

Figure 2. Vue globale existante du projet et de son ecosysteme

1.3 Problematique

- La gestion manuelle des clients et locations entraine des erreurs de ressaisie.
- Les paiements et depots sont difficiles a suivre sans historique centralise.
- Les documents ne sont pas toujours relies proprement au client ou a la location concernes.
- L import de donnees depuis Excel peut injecter des doublons si aucune validation n'est mise en place.
- La supervision et la securite sont insuffisantes sans audit, maintenance, permissions et seconde authentification.

1.4 Objectif general

Fournir une solution complete et moderne de gestion locative multi-roles, capable de gerer les clients, les locations, les paiements, les depots et les documents, avec un controle multi-niveaux, une traçabilite forte et une interface responsive sur web et desktop.

1.5 Objectifs specifiques

- Permettre au Super Admin de gerer les demandes admin, l activation des comptes, l impersonation et les politiques globales.
- Permettre a chaque admin de gerer uniquement ses clients, ses locations, ses paiements et ses documents.
- Automatiser l'import de donnees depuis Excel, CSV ou JSON avec mapping clair, validation, correction et journalisation.
- Mettre en place un controle de securite comprenant JWT, cookies, CSRF, audit, blocage IP, seconde authentification et gouvernance de maintenance.

- Permettre l'envoi de notifications transactionnelles claires et personnalisees vers les clients, les admins et les Super Admins.

Chapitre 2 - Analyse fonctionnelle et UML

2.1 Acteurs du systeme

Acteur	Responsabilites principales
Super Admin	Approuver les demandes admin, gouverner la plateforme, superviser l audit, activer la maintenance, valider certains paiements et impersoner un admin.
Admin	Gerer son portefeuille de clients, ses locations, ses paiements, ses depots, ses documents et ses imports.
Client	Recevoir des notifications et regulariser les paiements ou documents le concernant.
Services externes	Cloudinary pour les fichiers, Brevo pour les emails, providers de paiement pour les transactions et webhooks.

2.2 Stack technique et choix structurants

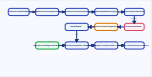
Composant	Choix
Frontend Web	React 18, Vite 7, TypeScript, Tailwind CSS, shadcn/ui
State Management	Zustand, React Context, TanStack Query

Composant	Choix
Routing	React Router DOM
Desktop	Electron 40
Backend	Next.js App Router, TypeScript, Zod
Architecture backend	POO / SOLID / Services / DAO / Prisma
Base de donnees	PostgreSQL via Prisma
Notifications	Brevo SMTP API, webhooks et ecouteurs
Media / fichiers	Cloudinary
PDF	jsPDF, html2canvas
Deploiement	Vercel (frontend), Render (backend), Docker / CI-CD cible

2.3 Diagrammes de cas d'utilisation

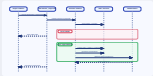
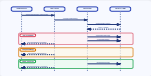
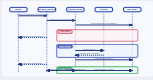
Figure 3. Use case global de l'application Figure 4. Use case dedie a l'import des clients

2.4 Diagrammes d activite

Figure 5. Activite d'approbation d'un administrateur  Figure 6. Activite d'import de clients Figure 7. Activite de paiement abonnement administrateur

2.5 Diagrammes de sequence

Les diagrammes de sequence suivants decrivent les interactions exactes entre les acteurs, le frontend, les routes API, les services metier et la persistance pour les fonctionnalites critiques.

 Figure 8. Sequence d'approbation administrateur  Figure 9. Sequence login et seconde authentification Figure 10. Sequence d'import clients  Figure 11. Sequence paiement abonnement administrateur

2.6 Exigences fonctionnelles majeures

- Gestion multi-roles avec isolation des donnees par administrateur et gouvernance globale cote Super Admin.
- Gestion complete du cycle client -> location -> paiements mensuels -> depots -> documents.
- Import de masse Excel / CSV / JSON avec mapping, validation, correction avant insertion et journal d'execution.
- Paiement d'abonnement administrateur avec verification du mois requis, gestion cash / mobile money, historisation et notification.
- Audit, maintenance, blocage IP, seconde authentification et configuration plateforme dynamique.

2.7 Exigences non fonctionnelles

- Responsive design sur mobile, tablette et desktop.
- Architecture maintenable et evolutive en couches frontend et backend.
- Performance acceptable sur les lectures et ecritures frequentes.

- Deploiement simple sur Vercel, Render ou environment Docker.
- Traçabilite forte des operations critiques via audit logs et undo actions.

Chapitre 3 - Methodologie et organisation

3.1 Approche de travail

Le projet a ete mene selon une logique iterative proche de l Agile, avec un decoupage court en fonctionnalites livrables. Chaque increment visait a produire une valeur visible: authentification, clients, locations, paiements, imports, gouvernance ou notifications. Cette approche a permis de corriger tres vite les problemes UX, les regressions et les blocages metier observes durant les tests.

3.2 Phasage du projet

Periode	Phase	Livrable
Jour 1 a 3	Cadrage fonctionnel	Definition des roles, du perimetre, des objets metier et des flux critiques.
Jour 4 a 6	Conception UML et architecture	Formalisation des cas d utilisation, activites, architecture frontend et backend.
Jour 7 a 12	Implementation coeur produit	Auth, clients, locations, paiements, documents, routes et store global.
Jour 13 a 16	Import et supervision	Import Excel/CSV/JSON, gestion des erreurs, import runs, audit, undo.

Periode	Phase	Livrable
Jour 17 a 19	Notifications et gouvernance	Brevo, relances, parametres Super Admin, maintenance, branding.
Jour 20 a 21	Tests et deploiement	Build, verifications, correction responsive et preparation soutenance.

3.3 Livrables attendus

- Application frontend fonctionnelle et responsive.
- Backend securise avec Prisma, routes, controleurs et services applicatifs.
- Documentation technique et architecture UML.
- Deploiement web avec configuration runtime de l API.
- Dossier de soutenance complet et preuves d implementation.

Chapitre 4 - Architecture technique frontend / backend

4.1 Architecture globale

L architecture globale de KYA repose sur une separation claire entre le frontend React/Vite, la couche de services frontend, le backend Next.js App Router, la logique applicative / domaine, la persistence PostgreSQL via Prisma et les integrations externes de notification et de media.

Figure 12. Architecture globale de la solution

4.2 Architecture frontend

Le frontend est structure en couches lisibles: routage principal, providers globaux, layout, pages, composants, services et infrastructure. Cette separation facilite les tests, la maintenance et la possibilite d executer le meme coeur produit dans un navigateur ou dans Electron.

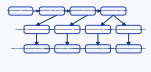


Figure 13. Architecture frontend

4.3 Architecture backend

Le backend suit une organisation POO / SOLID. Les routes App Router ne contiennent pas la logique metier; elles transmettent la requete vers un controleur. Les controleurs orchestrent les validateurs, DTO et services applicatifs. Les services s appuient ensuite sur des interfaces de persistence, implementees par des DAO et repositories Prisma ou memoire.

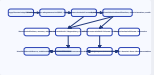


Figure 14. Architecture backend

4.4 Diagramme de deploiement

Figure 15. Architecture de deploiement

4.5 Cartographie des modules

Module	Description	Frontend	Backend
Authentification et session	Connexion admin / super admin, session	src/contexts/AuthContext.tsx, src/layouts/MainLayout.tsx	next-backend/app/api/authContext/*, next-backend/src/application

Module	Description	Frontend	Backend
	persistante, seconde authentification, impersonation		ervices/authentification/*
Gestion clients / locations	CRUD clients, locations, depots, filtres et ownership par admin	src/stores/dataStore.ts, src/pages/admin/clients/*, src/pages/admin/rentals/*	next-backend/app/api/clients next-backend/app/api/locatio /*, prisma/schema.prisma
Paiements	Paiements mensuels, recus PDF, cautions, statuts, tableaux de bord	src/pages/admin/payments/*, src/components/ReceiptModal.tsx	next-backend/app/api/ payments/*, next-backend/app/api/deposi *
Abonnement administrateur	Paiement abonnement, controle du mois requis, validation cash, notifications Super Admin	src/pages/admin/ AdminSubscriptionPayments.tsx	next-backend/app/api/ admin_payments/*, ne backend/app/api/admin_pay nts/notificationPaiementsAd n.ts
Documents	Upload PDF, archivage, contrats signes,	src/pages/admin/documents/*, src/lib/pdfUtils.ts	next-backend/app/api/ documents/*, next-backend/app/api/sign/ro

Module	Description	Frontend	Backend
	recus, partage		e.ts
Import de donnees	Analyse de fichier, mapping, validation, correction, preview editable, import runs et erreurs	src/pages/admin/imports/*	next-backend/app/api/import_runs/*
Notifications	Notifications admin, relances clients impayes, emails Brevo, webhooks	src/pages/super-admin/notifications/*	next-backend/app/api/notifications/*, next-backend/src/infrastructure/alertes/*
Gouvernance Super Admin	Maintenance, branding, politique session, webhook audit, configuration dynamique	src/pages/admin/misc/settings/*, src/services/platformConfig.ts	next-backend/app/api/settings/*, next-backend/src/infrastructure/http/politiquePlateforme.ts

Chapitre 5 - Modelisation des donnees et securite

5.1 Schema de donnees

Le modele relationnel retenu structure les comptes, les sessions, les permissions, les clients, les locations, les paiements, les depots, les documents, les imports, les notifications et l audit. Le schema Prisma garantit la centralisation et la durabilite des donnees metier cote backend.

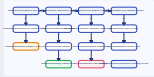


Figure 16. Modele de donnees principal

5.2 Entites coeur

- Utilisateur: identite, role, telephone, email, statut, seconde authentification Super Admin.
- Admin: permissions fines, statut, mode d'abonnement, rattachement entreprise.
- Client: identite, contacts, CNI, statut et relation avec les locations.
- Location: bien, loyer, date de debut, cautions, paiements mensuels, documents.
- ImportRun: fichier source, nombre de lignes, lignes inserees, erreurs, pages lues/non lues.
- Notification et JournalAudit: traçabilite des actions et diffusion des evenements.

5.3 Securite mise en place

- Extraction du jeton d acces depuis les cookies ou le bearer token.
- Protection CSRF sur les mutations.
- Blocage automatique des IP en fonction du nombre d'echecs de connexion.

- Seconde authentification Super Admin avec TOTP et TTL de verification.
- Impersonation encadree pour permettre au Super Admin de se faire passer pour un admin sans casser l isolation metier.
- Audit des operations sensibles et historique d annulation.

5.4 Configuration plateforme

Le Super Admin peut modifier a chaud les regles de maintenance, les durees de session, les politiques de paiement, les documents autorises, les canaux de notification, le branding et les exports d audit. Cette configuration est stockee dans la cle platform_config_v1 et rechargee cote frontend.

Lien	URL
Frontend web deploye	https://keur-ya-aicha-frontend.vercel.app
Backend API	https://bakend-next-saas-gestion-client.onrender.com/api
Documentation Swagger / docs	https://bakend-next-saas-gestion-client.onrender.com/docs
Portail documentation backend	https://bakend-next-saas-gestion-client.onrender.com/documentation

Chapitre 6 - Implementation detaillee des grandes fonctionnalites

6.1 Authentification, approbation et acces multi-roles

Le flux d authentification se base sur AuthContext cote frontend et sur un backend structure autour de l authContext, des services d authentification, du controle des permissions et des DAO Prisma. Apres approbation d une demande admin, le compte peut se connecter avec email ou telephone, y compris au format local ou prefixe +221.

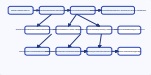


Figure 17. Diagramme de classes techniques

Le correctif de normalisation des telephones a ete important pour la stabilite de la connexion. Il evite qu un numero enregistre sous la forme +221771234567 soit considere comme invalide lorsque l utilisateur saisit 771234567.

6.2 Gestion clients, locations, paiements et documents

Le store Zustand centralise les actions metier du frontend. Les operations de creation client, ajout de location, versement mensuel, caution, upload de document et recalcul des statistiques y sont orchestrees. Le backend assure ensuite les verifications d ownership et la persistence relationnelle.

6.3 Import Excel / CSV / JSON

L import a constitue un axe majeur du projet. L administrateur peut charger un fichier Excel, CSV ou JSON, mapper les colonnes detectees, corriger les champs obligatoires,

previsualiser les lignes, lancer l analyse puis importer uniquement les lignes valides. En cas d erreur, un import run est cree avec les erreurs ligne par ligne et une navigation vers la page dediee.

- Lecture du fichier et extraction des en-tetes.
- Mapping manuel ou semi-automatique des colonnes.
- Validation des champs obligatoires et des formats.
- Correction inline des donnees avant insertion.
- Insertion des lignes valides avec timeout et progression.
- Creation d'un journal d'import (import run) pour les succes et les erreurs.

6.4 Paiement abonnement administrateur

La page d abonnement admin permet de payer le mois requis par Wave, Orange Money ou cash. Le backend controle les doublons, l ordre des mois, les restrictions de methode et la finalisation du paiement. Lorsqu un paiement devient effectivement paid, un evenement est publie pour notifier le Super Admin.

6.5 Notifications transactionnelles Brevo

Le service Brevo implemente une interface de notification generique. Il prend en charge la construction de templates HTML riches selon l evenement: creation de demande admin, approbation admin, paiement abonnement admin, retard de paiement client, etc. Le service d evenements permet d ajouter plusieurs ecouteurs sans coupler la logique metier a un provider unique.

6.6 Gouvernance plateforme et supervision

Le module de gouvernance expose des parametres transverses: maintenance, session, documents, paiements, notifications, branding, audit et conformite. Il alimente le

chargement runtime du frontend, conditionne le comportement de certains middlewares backend et facilite la supervision globale de la plateforme.

Chapitre 7 - Deploiement, tests et exploitation

7.1 Strategie de deploiement

Le frontend web est deploye sur Vercel. Le backend Next.js est deploye separement sur une plateforme de type Render, avec PostgreSQL comme base de donnees. Cette separation permet de conserver un backend persistant et de laisser le frontend Vite beneficier des avantages de Vercel pour le build et la distribution SPA.

Element	Valeur
Frontend	https://keur-ya-aicha-frontend.vercel.app
Backend API	https://bakend-next-saas-gestion-client.onrender.com/api
Docs backend	https://bakend-next-saas-gestion-client.onrender.com/docs
Commande build frontend	npm run build
Commande build backend	npm run build
Commande qualite backend	npm run lint && npm run typecheck
Build Electron Linux	npm run build:electron:linux

7.2 Variables d environnement importantes

Variable	Description
VITE_API_URL	URL publique du backend pour le frontend Vite
VITE_CLOUDINARY_SIGN_URL	URL de signature backend pour Cloudinary
DATABASE_URL	Connexion PostgreSQL Prisma
BREVO_API_KEY	Cle API Brevo pour les notifications transactionnelles
BREVO_SENDER_EMAIL	Expediteur email transactionnel
BREVO_NOTIFICATION_SUPER_ADMIN_EMAILS	Liste des destinataires Super Admin
ALERTE_SUPER_ADMIN_WEBHOOK_URL	Webhook alerte supervision

7.3 Tests et qualite

- Verification frontend par build Vite.
- Lint et typecheck sur le backend Next.
- Tests d endpoints disponibles cote backend pour les modules admin et Super Admin.
- Controle de responsive et de non debordement sur mobile.
- Verification des flux critiques: login, import, abonnement, notifications, erreurs et succes.

7.4 Exploitation

La version desktop Electron permet de conserver une experience hors navigateur, tout en gardant la possibilite de changer l URL de l API via un fichier runtime sans recompiler l application. Cette capacite est importante pour l installation sur plusieurs environnements.

Chapitre 8 - Discussion, limites et perspectives

8.1 Resultats obtenus

- Une plateforme web et desktop coherente pour la gestion locative multi-roles.
- Une separation claire entre gouvernance Super Admin et exploitation Admin.
- Un import de donnees plus simple a manipuler, corrigeable avant insertion et traçable.
- Une base de donnees relationnelle et un backend industrialisable en couches.
- Des notifications transactionnelles structurees et prêtes pour l exploitation reelle.

8.2 Limites actuelles

- Certaines integrations externes dependent des validations de compte chez les fournisseurs (par exemple Brevo transactionnel).
- Le mode mobile web est deja exploitable, mais certains modules secondaires peuvent encore etre harmonises pour obtenir une experience parfaitement uniforme.
- Une automatisation e2e plus poussee permettrait de produire encore plus de

preuves d execution sur les parcours critiques.

8.3 Perspectives

- Ajouter des tableaux de bord analytiques plus pousses pour les loyers, la rentabilite et les arrieres.
- Etendre les notifications vers WhatsApp et SMS selon les regles de gouvernance.
- Ajouter une gestion avancee des contrats, signatures et workflows documentaires.
- Mettre en place une pipeline CI/CD complete avec matrix build Electron multi-OS.
- Renforcer encore les tests e2e pour les parcours complets de gestion locative.

Conclusion generale

Le projet KYA montre qu une solution de gestion locative peut etre concue de maniere a la fois pratique pour les utilisateurs et rigoureuse du point de vue de l architecture. La plateforme ne se limite pas a un CRUD: elle integre une vraie gouvernance, des imports traces, des notifications, une securite renforcee, une compatibilite web / desktop et une base technique solide pour evoluer.

Du point de vue de la soutenance, la valeur du projet reside autant dans les fonctionnalites visibles que dans la structure interne du code: separation des responsabilites, centralisation des politiques, normalisation de l acces API, architecture backend en couches, schema Prisma, et prise en compte des contraintes de deploiement reelles.

En conclusion, KYA constitue une base serieuse de plateforme SaaS de gestion locative, exploitable, demonstrable et evolutive. Le present dossier apporte les pieces necessaires pour justifier les choix fonctionnels, techniques et architecturaux du projet.

Webographie et references

- Frontend deploye : https://keur-ya-aicha-frontend.vercel.app
- Backend API : https://bakend-next-saas-gestion-client.onrender.com/api
- Documentation backend : https://bakend-next-saas-gestion-client.onrender.com/docs
- Documentation interne du depot : docs/, docs/UML/, next-backend/docs/
- Schema Prisma : next-backend/prisma/schema.prisma

Annexes techniques

Annexe A - Arborescence frontend

```
src
src/App.css
src/App.tsx
src/components
src/components/AppSidebar.tsx
src/components/BadgeStatut.tsx
src/components/CardStat.tsx
src/components/ConfirmDialog.tsx
src/components/DepositModal.tsx
src/components/ErrorBoundary.tsx
src/components/FormError.tsx
src/components/LanguageSelector.tsx
src/components/NavLink.tsx
src/components/OfflineSyncBadge.tsx
src/components/PaymentModal.tsx
src/components/PrivateAdminRoute.tsx
src/components/QuickPaymentModal.tsx
src/components/Receipt.tsx
src/components/ReceiptModal.tsx
src/components/SearchInput.tsx
src/components/SendDownloadModal.tsx
src/components/Skeleton.tsx
src/components/ToastContainer.tsx
src/components/page_inscription
src/components/page_inscription/admin-signup
```

```
src/components/page_inscription/p
src/components/ui
src/components/ui/accordion.tsx
src/components/ui/alert-dialog.tsx
src/components/ui/alert.tsx
src/components/ui/aspect-ratio.tsx
src/components/ui/avatar.tsx
src/components/ui/badge.tsx
src/components/ui/breadcrumb.tsx
src/components/ui/button.tsx
src/components/ui/calendar.tsx
src/components/ui/card.tsx
src/components/ui/carousel.tsx
src/components/ui/chart.tsx
src/components/ui/checkbox.tsx
src/components/ui/collapsible.tsx
src/components/ui/command.tsx
src/components/ui/context-menu.tsx
src/components/ui/dialog.tsx
src/components/ui/drawer.tsx
src/components/ui/dropdown-menu.tsx
src/components/ui/form.tsx
src/components/ui/hover-card.tsx
src/components/ui/input-otp.tsx
src/components/ui/input.tsx
src/components/ui/label.tsx
src/components/ui/menubar.tsx
src/components/ui/navigation-menu.tsx
src/components/ui/pagination.tsx
src/components/ui/popover.tsx
src/components/ui/progress.tsx
src/components/ui/radio-group.tsx
src/components/ui/resizable.tsx
src/components/ui/scroll-area.tsx
src/components/ui/select.tsx
src/components/ui/separator.tsx
src/components/ui/sheet.tsx
src/components/ui/sidebar.tsx
src/components/ui/skeleton.tsx
src/components/ui/slider.tsx
src/components/ui/sonner.tsx
src/components/ui/switch.tsx
src/components/ui/table.tsx
src/components/ui/tabs.tsx
src/components/ui/textarea.tsx
src/components/ui/toast.tsx
src/components/ui/toaster.tsx
src/components/ui/toggle-group.tsx
src/components/ui/toggle.tsx
src/components/ui/tooltip.tsx
```

```
src/components/ui/use-toast.ts
src/contexts
src/contexts/AuthContext.tsx
src/contexts/ToastContext.tsx
src/dto
src/dto/backend
src/dto/backend/requests
src/dto/backend/responses
src/dto/frontend
src/dto/frontend/requests
src/dto/frontend/responses
src/hooks
src/hooks/index.ts
src/hooks/use-mobile.tsx
src/hooks/use-toast.ts
src/hooks/useAdminDashboard.ts
src/hooks/useApiCall.ts
src/hooks/useApiHandler.ts
src/hooks/useAsync.ts
src/hooks/useCloudinaryUpload.ts
src/hooks/useElectronAPI.ts
src/hooks/useGoBack.ts
src/hooks/useImportClients.ts
src/hooks/usePagination.ts
src/hooks/useSearch.ts
src/index.css
src/infrastructure
src/infrastructure/indexedDb.ts
src/infrastructure/offlineCache.ts
src/infrastructure/syncQueue.ts
src/layouts
src/layouts/MainLayout.tsx
src/lib
src/lib/__tests__
src/lib/__tests__/importClients.parseSpreadsheet.test.ts
src/lib/__tests__/importClients.test.ts
src/lib/actionLogger.ts
src/lib/constants
src/lib/constants.ts
src/lib/constants/colors.ts
src/lib/constants/errors.ts
src/lib/constants/index.ts
src/lib/documentDisplay.ts
src/lib/domMutationGuard.ts
src/lib/formatters.ts
src/lib/i18n.tsx
src/lib/importClients.ts
src/lib/pdfUtils.ts
src/lib/types.ts
src/lib/utils.ts
```

```
src/login.css
src/login.html
src/main.tsx
src/messages
src/messages/anglais
src/messages/anglais/validation.auth.ts
src/messages/anglais/validation.client.ts
src/messages/anglais/validation.common.ts
src/messages/anglais/validation.payment.ts
src/messages/anglais/validation.property.ts
src/messages/anglais/validation.ts
src/messages/app
src/messages/app.ts
src/messages/app/actions.ts
src/messages/app/confirmations.ts
src/messages/app/entities.ts
src/messages/app/errors.ts
src/messages/app/index.ts
src/messages/app/labels.ts
src/messages/app/menu.ts
src/messages/app/nav.ts
src/messages/app/pages.ts
src/messages/app/status.ts
src/messages/app/success.ts
src/messages/app/validation.ts
src/messages/franchais
src/messages/franchais/validation.auth.ts
src/messages/franchais/validation.client.ts
src/messages/franchais/validation.common.ts
src/messages/franchais/validation.payment.ts
src/messages/franchais/validation.property.ts
src/messages/franchais/validation.ts
src/messages/index.ts
src/messages/validation-keys.ts
src/messages/validation.ts
src/pages
src/pages/admin
src/pages/admin/AddClient.tsx
src/pages/admin/AddPayment.tsx
src/pages/admin/AddRental.tsx
src/pages/admin/AdminSubscriptionPayments.tsx
src/pages/admin/Archive.tsx
src/pages/admin/ArchivedClients.tsx
src/pages/admin/BlacklistedClients.tsx
src/pages/admin/ClientDetail.tsx
src/pages/admin/ClientDossier.tsx
src/pages/admin/Clients.tsx
src/pages/admin/DangerClients.tsx
src/pages/admin/Dashboard.tsx
src/pages/admin/Deposits.tsx
```

```
src/pages/admin/Documents.tsx
src/pages/admin/EditRental.tsx
src/pages/admin/ImportClients.tsx
src/pages/admin/ImportErrors.tsx
src/pages/admin/ImportSuccess.tsx
src/pages/admin/PaymentHistory.tsx
src/pages/admin/PaymentReceipts.tsx
src/pages/admin/Payments.tsx
src/pages/admin/RentalDetail.tsx
src/pages/admin/Rentals.tsx
src/pages/admin/Settings.tsx
src/pages/admin/SignedContracts.tsx
src/pages/admin/Work.ts
src/pages/admin/archive
src/pages/admin/clients
src/pages/admin/documents
src/pages/admin/imports
src/pages/admin/misc
src/pages/admin/payments
src/pages/admin/rentals
src/pages/common
src/pages/common/AdminSignup.tsx
src/pages/common/AuthLoginCard.tsx
src/pages/common/ForbiddenMessage.tsx
src/pages/common/Index.tsx
src/pages/common/Login.test.tsx
src/pages/common/Login.tsx
src/pages/common/NotFound.tsx
src/pages/common/PageHeader.tsx
src/pages/common/SectionWrapper.tsx
src/pages/common/StatsCards.tsx
src/pages/common/index.ts
src/pages/super-admin
src/pages/super-admin/Entreprises.tsx
src/pages/super-admin/ForbiddenMessage.tsx
src/pages/super-admin/SuperAdminDashboard.tsx
src/pages/super-admin/SuperAdminLogin.tsx
src/pages/super-admin/SuperAdminPage.tsx
src/pages/super-admin/SuperAdminStats.tsx
src/pages/super-admin/admins
src/pages/super-admin/components
src/pages/super-admin/constants.ts
src/pages/super-admin/core
src/pages/super-admin/entreprises
src/pages/super-admin/forms
src/pages/super-admin/index.ts
src/pages/super-admin/logs
src/pages/super-admin/monitoring
src/pages/super-admin/notifications
src/pages/super-admin/sections
```

```
src/pages/super-admin/settings
src/pages/super-admin/stats
src/pages/super-admin/types.ts
src/pages/super-admin/utils.ts
src/services
src/services/__tests__
src/services/__tests__/api.test.ts
src/services/adminBranding.ts
src/services/adminPermissions.ts
src/services/api
src/services/api.ts
src/services/api/adminPayments.api.ts
src/services/api/admins.api.ts
src/services/api/auditLogs.api.ts
src/services/api/auth.api.ts
src/services/api/blockedIps.api.ts
src/services/api/clients.api.ts
src/services/api/deposits.api.ts
src/services/api/documents.api.ts
src/services/api/endpoint.factory.ts
src/services/api/entreprises.api.ts
src/services/api/importRuns.api.ts
src/services/api/notifications.api.ts
src/services/api/payments.api.ts
src/services/api/settings.api.ts
src/services/api/undo.api.ts
src/services/api/uploads.api.ts
src/services/api/users.api.ts
src/services/api/workItems.api.ts
src/services/assets.ts
src/services/data
src/services/data/normalizers.ts
src/services/data/serialization.ts
src/services/data/stats.ts
src/services/data/transform.ts
src/services/http.ts
src/services/importErrors.ts
src/services/platformConfig.ts
src/services/runtimeConfig.ts
src/services/services
src/services/services/deposits.service.ts
src/services/services/payments.service.ts
src/services/uploader
src/services/uploader/cloudinary.uploader.ts
src/services/uploader/file-uploader.interface.ts
src/services/uploader/uploader.factory.ts
src/services/utils
src/services/utils/ids.ts
src/stores
src/stores/DataProvider.tsx
```

```
src/stores/dataStore.ts
src/test
src/test/example.test.ts
src/test/setup.ts
src/utils
src/utils/apiFieldErrors.ts
src/validators
src/validators/__tests__
src/validators/__tests__/clientValidator.test.ts
src/validators/backend
src/validators/backend/common.ts
src/validators/backend/index.ts
src/validators/frontend
src/validators/frontend/auth
src/validators/frontend/client
src/validators/frontend/index.ts
src/validators/frontend/payment
src/validators/frontend/rental
src/vite-env.d.ts
```

Annexe B - Arborescence backend

```
next-backend/src
next-backend/src/application
next-backend/src/application/dtos
next-backend/src/application/dtos/administration
next-backend/src/application/dtos/administration/DtoAdministration.ts
next-backend/src/application/dtos/administration/index.ts
next-backend/src/application/dtos/authentification
next-backend/src/application/dtos/authentification/DtoAuthentification.ts
next-backend/src/application/dtos/authentification/index.ts
next-backend/src/application/dtos/index.ts
next-backend/src/application/dtos/locations
next-backend/src/application/dtos/locations/DtoLocations.ts
next-backend/src/application/dtos/locations/index.ts
next-backend/src/application/dtos/systeme
next-backend/src/application/dtos/systeme/DtoSysteme.ts
next-backend/src/application/dtos/systeme/index.ts
next-backend/src/application/dtos/utilisateurs
next-backend/src/application/dtos/utilisateurs/DtoUtilisateurs.ts
next-backend/src/application/dtos/utilisateurs/index.ts
next-backend/src/application/exceptions
next-backend/src/application/exceptions/ExceptionApplication.ts
next-backend/src/application/exceptions/ExceptionAuthentification.ts
next-backend/src/application/exceptions/ExceptionAuthentificationAutorisation.ts
next-backend/src/application/exceptions/ExceptionAuthentificationLimiteTentatives.ts
next-backend/src/application/exceptions/ExceptionAuthentificationPrecondition.ts
```

```
next-backend/src/application/exceptions/ExceptionAuthentificationValidation.ts
next-backend/src/application/exceptions/ExceptionCasUsage.ts
next-backend/src/application/exceptions/ExceptionValidationApplication.ts
next-backend/src/application/exceptions/index.ts
next-backend/src/application/fabriques
next-backend/src/application/fabriques/FabriqueJetonRefresh.ts
next-backend/src/application/fabriques/FabriqueSessionAuthentification.ts
next-backend/src/application/fabriques/FabriqueUtilisateur.ts
next-backend/src/application/fabriques/index.ts
next-backend/src/application/mappers
next-backend/src/application/mappers/MappeurUtilisateur.ts
next-backend/src/application/mappers/MappeurUtilisateurAuthentification.ts
next-backend/src/application/mappers/administration
next-backend/src/application/mappers/administration/MappeurAdministrationAdmin.ts
next-backend/src/application/mappers/administration/MappeurAdministrationSupervision.ts
next-backend/src/application/mappers/administration/index.ts
next-backend/src/application/mappers/index.ts
next-backend/src/application/services
next-backend/src/application/services/administration
next-backend/src/application/services/administration/ServiceAdministrationAdmin.ts
next-backend/src/application/services/administration/audit
next-backend/src/application/services/administration/clients
next-backend/src/application/services/administration/commun
next-backend/src/application/services/administration/constructeurs
next-backend/src/application/services/administration/documents
next-backend/src/application/services/administration/index.ts
next-backend/src/application/services/administration/notifications
next-backend/src/application/services/administration/paiements
next-backend/src/application/services/administration/supervision
next-backend/src/application/services/administration/travaux
next-backend/src/application/services/authentification
next-backend/src/application/services/authentification/ServiceAuditAuthentification.ts
next-backend/src/application/services/authentification/ServiceAuthentification.ts
next-backend/src/application/services/authentification/ServiceAutorisationAuthentification.ts
next-backend/src/application/services/authentification/ServiceContexteAuthentification.ts
next-backend/src/application/services/authentification/ServiceImpersonationAuthentification.ts
next-backend/src/application/services/authentification/
ServiceSecuriteSessionAuthentification.ts
next-backend/src/application/services/authentification/ServiceSessionAuthentification.ts
next-backend/src/application/services/authentification/ServiceTotpSuperAdminAuthentification.ts
next-backend/src/application/services/authentification/index.ts
next-backend/src/application/services/index.ts
next-backend/src/application/services/sante
next-backend/src/application/services/sante/ServiceSante.ts
next-backend/src/application/services/sante/index.ts
next-backend/src/application/types
next-backend/src/application/types/administration
next-backend/src/application/types/administration/TypeDependancesServiceAdministrationAdmin.ts
next-backend/src/application/types/administration/audit
next-backend/src/application/types/administration/clients
```

```
next-backend/src/application/types/administration/commun
next-backend/src/application/types/administration/documents
next-backend/src/application/types/administration/index.ts
next-backend/src/application/types/administration/notifications
next-backend/src/application/types/administration/paiements
next-backend/src/application/types/administration/supervision
next-backend/src/application/types/administration/travaux
next-backend/src/application/types/index.ts
next-backend/src/application/validateurs
next-backend/src/application/validateurs/ValidateurAuthentification.ts
next-backend/src/application/validateurs/ValidateurUtilisateur.ts
next-backend/src/coeur
next-backend/src/coeur/configuration
next-backend/src/coeur/configuration/ConfigurationApplication.ts
next-backend/src/coeur/configuration/ConfigurationSecurite.ts
next-backend/src/coeur/conteneur
next-backend/src/coeur/conteneur/ConteneurDependances.ts
next-backend/src/coeur/conteneur/FabriqueServiceAdministrationAdminSupervision.ts
next-backend/src/coeur/erreurs
next-backend/src/coeur/erreurs/ErreurHttp.ts
next-backend/src/coeur/exceptions
next-backend/src/coeur/exceptions/ExceptionBase.ts
next-backend/src/coeur/exceptions/ExceptionConfiguration.ts
next-backend/src/coeur/exceptions/ExceptionHttp.ts
next-backend/src/coeur/exceptions/ExceptionTechnique.ts
next-backend/src/coeur/exceptions/index.ts
next-backend/src/coeur/interfaces
next-backend/src/coeur/interfaces/InterfaceClientBaseDeDonnees.ts
next-backend/src/coeur/interfaces/InterfaceNotification.ts
next-backend/src/coeur/interfaces/InterfaceServiceAuditSecurite.ts
next-backend/src/coeur/interfaces/InterfaceServiceChiffrement.ts
next-backend/src/coeur/interfaces/InterfaceServiceHachageMotDePasse.ts
next-backend/src/coeur/interfaces/InterfaceServiceJetonAcces.ts
next-backend/src/coeur/interfaces/InterfaceServiceTotp.ts
next-backend/src/coeur/interfaces/InterfaceUtilitairesSecurite.ts
next-backend/src/coeur/interfaces/InterfaceValidateurAuthentification.ts
next-backend/src/coeur/interfaces/InterfaceValidateurEntree.ts
next-backend/src/controleurs
next-backend/src/controleurs/ControleurAdministrationAdmin.ts
next-backend/src/controleurs/ControleurAuthContext.ts
next-backend/src/controleurs/ControleurSante.ts
next-backend/src/docs
next-backend/src/docs/README.ts
next-backend/src/documentation
next-backend/src/documentation/GenerateurSwagger.ts
next-backend/src/documentation/SwaggerAdministrationAdmin.ts
next-backend/src/domaine
next-backend/src/domaine/builders
next-backend/src/domaine/builders/BuilderAbstrait.ts
next-backend/src/domaine/builders/administration
```

```
next-backend/src/domaine/builders/administration/BuilderEntiteAdmin.ts
next-backend/src/domaine/builders/administration/BuilderEntiteBrandingAdmin.ts
next-backend/src/domaine/builders/administration/BuilderEntiteDemandeAdmin.ts
next-backend/src/domaine/builders/administration/BuilderEntiteEntreprise.ts
next-backend/src/domaine/builders/administration/BuilderEntitePaiementAbonnementAdmin.ts
next-backend/src/domaine/builders/administration/BuilderEntitePermissionsAdmin.ts
next-backend/src/domaine/builders/administration/BuilderEntiteStatutAbonnementAdmin.ts
next-backend/src/domaine/builders/administration/index.ts
next-backend/src/domaine/builders/authentification
next-backend/src/domaine/builders/authentification/
BuilderEntiteAuditSecuriteAuthentification.ts
next-backend/src/domaine/builders/authentification/BuilderEntiteJetonRefresh.ts
next-backend/src/domaine/builders/authentification/BuilderEntitePermissionUtilisateurAuth.ts
next-backend/src/domaine/builders/authentification/BuilderEntiteSessionAuthentification.ts
next-backend/src/domaine/builders/authentification/BuilderEntiteTentativeConnexion.ts
next-backend/src/domaine/builders/authentification/BuilderEntiteUtilisateurAuthentification.ts
next-backend/src/domaine/builders/authentification/index.ts
next-backend/src/domaine/builders/index.ts
next-backend/src/domaine/builders/locations
next-backend/src/domaine/builders/locations/BuilderEntiteCaution.ts
next-backend/src/domaine/builders/locations/BuilderEntiteClient.ts
next-backend/src/domaine/builders/locations/BuilderEntiteDocument.ts
next-backend/src/domaine/builders/locations/BuilderEntiteLocation.ts
next-backend/src/domaine/builders/locations/BuilderEntitePaiementCaution.ts
next-backend/src/domaine/builders/locations/BuilderEntitePaiementMensuel.ts
next-backend/src/domaine/builders/locations/BuilderEntiteTransactionPaiement.ts
next-backend/src/domaine/builders/locations/index.ts
next-backend/src/domaine/builders/systeme
next-backend/src/domaine/builders/systeme/BuilderEntiteConfigurationPlateforme.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteErreurImport.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteExecutionImport.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteIpBloquee.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteItemTravail.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteJournalAudit.ts
next-backend/src/domaine/builders/systeme/BuilderEntiteNotification.ts
next-backend/src/domaine/builders/systeme/index.ts
next-backend/src/domaine/builders/utilisateurs
next-backend/src/domaine/builders/utilisateurs/BuilderEntiteUtilisateur.ts
next-backend/src/domaine/builders/utilisateurs/index.ts
next-backend/src/domaine/entites
next-backend/src/domaine/entites/ObjetDomaine.ts
next-backend/src/domaine/entites/administration
next-backend/src/domaine/entites/administration/EntiteAdmin.ts
next-backend/src/domaine/entites/administration/EntiteBrandingAdmin.ts
next-backend/src/domaine/entites/administration/EntiteDemandeAdmin.ts
next-backend/src/domaine/entites/administration/EntiteEntreprise.ts
next-backend/src/domaine/entites/administration/EntitePaiementAbonnementAdmin.ts
next-backend/src/domaine/entites/administration/EntitePermissionsAdmin.ts
next-backend/src/domaine/entites/administration/EntiteStatutAbonnementAdmin.ts
next-backend/src/domaine/entites/administration/index.ts
```

```
next-backend/src/domaine/entites/authentification
next-backend/src/domaine/entites/authentification/EntiteAuditSecuriteAuthentification.ts
next-backend/src/domaine/entites/authentification/EntiteJetonRefresh.ts
next-backend/src/domaine/entites/authentification/EntitePermissionUtilisateurAuth.ts
next-backend/src/domaine/entites/authentification/EntiteSessionAuthentification.ts
next-backend/src/domaine/entites/authentification/EntiteTentativeConnexion.ts
next-backend/src/domaine/entites/authentification/EntiteUtilisateurAuthentification.ts
next-backend/src/domaine/entites/authentification/index.ts
next-backend/src/domaine/entites/index.ts
next-backend/src/domaine/entites/locations
next-backend/src/domaine/entites/locations/EntiteCaution.ts
next-backend/src/domaine/entites/locations/EntiteClient.ts
next-backend/src/domaine/entites/locations/EntiteDocument.ts
next-backend/src/domaine/entites/locations/EntiteLocation.ts
next-backend/src/domaine/entites/locations/EntitePaiementCaution.ts
next-backend/src/domaine/entites/locations/EntitePaiementMensuel.ts
next-backend/src/domaine/entites/locations/EntiteTransactionPaiement.ts
next-backend/src/domaine/entites/locations/index.ts
next-backend/src/domaine/entites/systeme
next-backend/src/domaine/entites/systeme/EntiteConfigurationPlateforme.ts
next-backend/src/domaine/entites/systeme/EntiteErreurImport.ts
next-backend/src/domaine/entites/systeme/EntiteExecutionImport.ts
next-backend/src/domaine/entites/systeme/EntiteIpBloquee.ts
next-backend/src/domaine/entites/systeme/EntiteItemTravail.ts
next-backend/src/domaine/entites/systeme/EntiteJournalAudit.ts
next-backend/src/domaine/entites/systeme/EntiteNotification.ts
next-backend/src/domaine/entites/systeme/index.ts
next-backend/src/domaine/entites/utilisateurs
next-backend/src/domaine/entites/utilisateurs/EntiteUtilisateur.ts
next-backend/src/domaine/entites/utilisateurs/index.ts
next-backend/src/domaine/enumerations
next-backend/src/domaine/enumerations/EnumerationRoleUtilisateur.ts
next-backend/src/domaine/exceptions
next-backend/src/domaine/exceptions/ExceptionDomaine.ts
next-backend/src/domaine/exceptions/ExceptionEntiteInvalide.ts
next-backend/src/domaine/exceptions/ExceptionObjetValeurInvalide.ts
next-backend/src/domaine/exceptions/ExceptionRegleMetier.ts
next-backend/src/domaine/exceptions/index.ts
next-backend/src/domaine/interfaces
next-backend/src/domaine/interfaces/InterfaceReferentielUtilisateur.ts
next-backend/src/domaine/interfaces/dao
next-backend/src/domaine/interfaces/dao/administration
next-backend/src/domaine/interfaces/dao/authentification
next-backend/src/domaine/interfaces/dao/index.ts
next-backend/src/domaine/interfaces/dao/locations
next-backend/src/domaine/interfaces/dao/systeme
next-backend/src/domaine/interfaces/dao/utilisateurs
next-backend/src/domaine/interfaces/repository
next-backend/src/domaine/interfaces/repository/InterfaceRepositoryAuthentification.ts
next-backend/src/domaine/interfaces/repository/index.ts
```

```
next-backend/src/domaine/objets_valeur
next-backend/src/domaine/objets_valeur/administration
next-backend/src/domaine/objets_valeur/administration/ObjetValeurCouleurHexadecimale.ts
next-backend/src/domaine/objets_valeur/administration/ObjetValeurMoisComptable.ts
next-backend/src/domaine/objets_valeur/administration/index.ts
next-backend/src/domaine/objets_valeur/authentification
next-backend/src/domaine/objets_valeur/authentification/ObjetValeurAdresseIp.ts
next-backend/src/domaine/objets_valeur/authentification/ObjetValeurAgentUtilisateur.ts
next-backend/src/domaine/objets_valeur/authentification/ObjetValeurHachageJetonRefresh.ts
next-backend/src/domaine/objets_valeur/authentification/ObjetValeurIdentifiantConnexion.ts
next-backend/src/domaine/objets_valeur/authentification/ObjetValeurJetonAccesJti.ts
next-backend/src/domaine/objets_valeur/authentification/index.ts
next-backend/src/domaine/objets_valeur/commun
next-backend/src/domaine/objets_valeur/commun/ObjetValeurBooleen.ts
next-backend/src/domaine/objets_valeur/commun/ObjetValeurEmail.ts
next-backend/src/domaine/objets_valeur/commun/ObjetValeurIdentifiant.ts
next-backend/src/domaine/objets_valeur/commun/ObjetValeurMontant.ts
next-backend/src/domaine/objets_valeur/commun/ObjetValeurTexteNonVide.ts
next-backend/src/domaine/objets_valeur/commun/ObjetValeurUrlHttpOuChemin.ts
next-backend/src/domaine/objets_valeur/commun/index.ts
next-backend/src/domaine/objets_valeur/index.ts
next-backend/src/domaine/objets_valeur/locations
next-backend/src/domaine/objets_valeur/locations/ObjetValeurCniSenegal.ts
next-backend/src/domaine/objets_valeur/locations/ObjetValeurNumeroRecu.ts
next-backend/src/domaine/objets_valeur/locations/ObjetValeurTelephoneSenegal.ts
next-backend/src/domaine/objets_valeur/locations/index.ts
next-backend/src/domaine/types
next-backend/src/domaine/types/administration
next-backend/src/domaine/types/administration/TypeActionAnnulationInterneAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeContexteAccesAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeEnregistrementParametreAdministrationAdmin.ts
next-backend/src/domaine/types/administration/
TypeMetadonneesActionAnnulationAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeMethodePaiementAbonnement.ts
next-backend/src/domaine/types/administration/TypeModeAbonnementAdmin.ts
next-backend/src/domaine/types/administration/TypeOperationAnnulationAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeOrdreTriAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeRessourceAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeResultatMutationAdministrationAdmin.ts
next-backend/src/domaine/types/administration/TypeStatutAdmin.ts
next-backend/src/domaine/types/administration/TypeStatutMouvement.ts
next-backend/src/domaine/types/administration/TypeStatutPaiementAbonnement.ts
next-backend/src/domaine/types/administration/TypeStatutPaiementAdminDto.ts
next-backend/src/domaine/types/administration/index.ts
next-backend/src/domaine/types/authentification
next-backend/src/domaine/types/authentification/TypeCommandeCreationSessionAuthentification.ts
next-backend/src/domaine/types/authentification/
TypeCommandeRotationJetonRefreshAuthentification.ts
next-backend/src/domaine/types/authentification/TypeContexteRequeteAuthentification.ts
next-backend/src/domaine/types/authentification/TypeContexteSessionAuthentification.ts
```

```
next-backend/src/domaine/types/authentification/TypeDriverPersistanceAuthentification.ts
next-backend/src/domaine/types/authentification/TypeEtatImpersonationAuthentification.ts
next-backend/src/domaine/types/authentification/TypeResultatConnexionAuthentification.ts
next-backend/src/domaine/types/authentification/TypeResultatRafraichissementAuthentification.ts
next-backend/src/domaine/types/authentification/
TypeResultatTotpInitialisationAuthentification.ts
next-backend/src/domaine/types/authentification/TypeTentativeConnexionAuthentification.ts
next-backend/src/domaine/types/authentification/TypeTotpSuperAdminAuthentification.ts
next-backend/src/domaine/types/authentification/index.ts
next-backend/src/domaine/types/index.ts
next-backend/src/domaine/types/locations
next-backend/src/domaine/types/locations/TypeBien.ts
next-backend/src/domaine/types/locations/TypeDocument.ts
next-backend/src/domaine/types/locations/TypeStatutClient.ts
next-backend/src/domaine/types/locations/TypeStatutPaiementMensuel.ts
next-backend/src/domaine/types/locations/index.ts
next-backend/src/domaine/types/systeme
next-backend/src/domaine/types/systeme/TypeLigneImportee.ts
next-backend/src/domaine/types/systeme/TypePrioriteTravail.ts
next-backend/src/domaine/types/systeme/TypeStatutTravail.ts
next-backend/src/domaine/types/systeme/TypesConfigurationNotifications.ts
next-backend/src/domaine/types/systeme/index.ts
next-backend/src/domaine/types/utilisateurs
next-backend/src/domaine/types/utilisateurs/TypeStatutUtilisateur.ts
next-backend/src/domaine/types/utilisateurs/index.ts
next-backend/src/infrastructure
next-backend/src/infrastructure/alertes
next-backend/src/infrastructure/alertes/ServiceAlerteSuperAdminWebhook.ts
next-backend/src/infrastructure/alertes/ServiceEcouteurEvenementsNotificationBrevo.ts
next-backend/src/infrastructure/alertes/ServiceEcouteurEvenementsNotificationWebhook.ts
next-backend/src/infrastructure/alertes/ServiceEmailBrevo.ts
next-backend/src/infrastructure/alertes/ServiceEvenementsNotification.ts
next-backend/src/infrastructure/base_de_donnees
next-backend/src/infrastructure/base_de_donnees/AdaptateurPrisma.ts
next-backend/src/infrastructure/base_de_donnees/ClientPrisma.ts
next-backend/src/infrastructure/dao
next-backend/src/infrastructure/dao/index.ts
next-backend/src/infrastructure/dao/memoire
next-backend/src/infrastructure/dao/memoire/administration
next-backend/src/infrastructure/dao/memoire/authentification
next-backend/src/infrastructure/dao/memoire/index.ts
next-backend/src/infrastructure/dao/memoire/locations
next-backend/src/infrastructure/dao/memoire/systeme
next-backend/src/infrastructure/dao/memoire/utilisateurs
next-backend/src/infrastructure/dao/prisma
next-backend/src/infrastructure/dao/prisma/administration
next-backend/src/infrastructure/dao/prisma/authentification
next-backend/src/infrastructure/dao/prisma/commun
next-backend/src/infrastructure/dao/prisma/index.ts
next-backend/src/infrastructure/dao/prisma/locations
```

```
next-backend/src/infrastructure/dao/prisma/systeme
next-backend/src/infrastructure/dao/prisma/utilisateurs
next-backend/src/infrastructure/exceptions
next-backend/src/infrastructure/exceptions/ExceptionAccesDonnees.ts
next-backend/src/infrastructure/exceptions/ExceptionDependanceExterne.ts
next-backend/src/infrastructure/exceptions/ExceptionInfrastructure.ts
next-backend/src/infrastructure/exceptions/index.ts
next-backend/src/infrastructure/http
next-backend/src/infrastructure/http/ContexteRequeteHttp.ts
next-backend/src/infrastructure/http/ReponseHttp.ts
next-backend/src/infrastructure/http/adapterUtilisateurAuthentifieFrontend.ts
next-backend/src/infrastructure/http/appliquerEntetesAnnulation.ts
next-backend/src/infrastructure/http/executerAvecGestionErreurs.ts
next-backend/src/infrastructure/http/executerMutationIdempotente.ts
next-backend/src/infrastructure/http/politiquePlateforme.ts
next-backend/src/infrastructure/http/verifierBlocageAbonnementGlobal.ts
next-backend/src/infrastructure/http/verifierMaintenanceGlobale.ts
next-backend/src/infrastructure/middlewares
next-backend/src/infrastructure/middlewares/InterfaceMiddlewareHttp.ts
next-backend/src/infrastructure/middlewares/MiddlewareGlobal.ts
next-backend/src/infrastructure/middlewares/MiddlewareJournalisation.ts
next-backend/src/infrastructure/middlewares/MiddlewareMaintenance.ts
next-backend/src/infrastructure/middlewares/PipelineMiddlewaresHttp.ts
next-backend/src/infrastructure/middlewares/index.ts
next-backend/src/infrastructure/repositories
next-backend/src/infrastructure/repositories/RepositoryAuthentificationAbstrait.ts
next-backend/src/infrastructure/repositories/index.ts
next-backend/src/infrastructure/repositories/memoire
next-backend/src/infrastructure/repositories/memoire/RepositoryAuthentificationMemoire.ts
next-backend/src/infrastructure/repositories/memoire/index.ts
next-backend/src/infrastructure/repositories/prisma
next-backend/src/infrastructure/repositories/prisma/RepositoryAuthentificationPrisma.ts
next-backend/src/infrastructure/repositories/prisma/index.ts
next-backend/src/infrastructure/securite
next-backend/src/infrastructure/securite/AdaptateurRequeteSecurite.ts
next-backend/src/infrastructure/securite/ServiceAuditSecurite.ts
next-backend/src/infrastructure/securite/ServiceAuditSecuriteMemoire.ts
next-backend/src/infrastructure/securite/ServiceChiffrementSymetrique.ts
next-backend/src/infrastructure/securite/ServiceCookiesAuthentification.ts
next-backend/src/infrastructure/securite/ServiceCorsStrict.ts
next-backend/src/infrastructure/securite/ServiceEntetesSecuriteHttp.ts
next-backend/src/infrastructure/securite/ServiceHachageMotDePasseArgon2.ts
next-backend/src/infrastructure/securite/ServiceJetonAccesJwt.ts
next-backend/src/infrastructure/securite/ServiceProtectionCsrf.ts
next-backend/src/infrastructure/securite/ServiceTotp.ts
next-backend/src/infrastructure/securite/UtilitairesSecurite.ts
next-backend/src/infrastructure/utils
next-backend/src/infrastructure/utils/UtilitaireDate.ts
next-backend/src/infrastructure/utils/UtilitaireIdentifiant.ts
next-backend/src/infrastructure/validateurs
```

```
next-backend/src/infrastructure/validateurs/ValidateurAuthentificationZod.ts
next-backend/src/infrastructure/validateurs/ValidateurZod.ts
next-backend/src/messages
next-backend/src/messages/anglais
next-backend/src/messages/anglais/validation.auth.ts
next-backend/src/messages/anglais/validation.client.ts
next-backend/src/messages/anglais/validation.common.ts
next-backend/src/messages/anglais/validation.payment.ts
next-backend/src/messages/anglais/validation.property.ts
next-backend/src/messages/anglais/validation.ts
next-backend/src/messages/app
next-backend/src/messages/app.ts
next-backend/src/messages/app/actions.ts
next-backend/src/messages/app/administration.ts
next-backend/src/messages/app/code.http.ts
next-backend/src/messages/app/confirmations.ts
next-backend/src/messages/app/entities.ts
next-backend/src/messages/app/errors.ts
next-backend/src/messages/app/index.ts
next-backend/src/messages/app/labels.ts
next-backend/src/messages/app/menu.ts
next-backend/src/messages/app/nav.ts
next-backend/src/messages/app/pages.ts
next-backend/src/messages/app/status.ts
next-backend/src/messages/app/success.ts
next-backend/src/messages/app/validation.ts
next-backend/src/messages/franchais
next-backend/src/messages/franchais/validation.auth.ts
next-backend/src/messages/franchais/validation.client.ts
next-backend/src/messages/franchais/validation.common.ts
next-backend/src/messages/franchais/validation.payment.ts
next-backend/src/messages/franchais/validation.property.ts
next-backend/src/messages/franchais/validation.ts
next-backend/src/messages/index.ts
next-backend/src/messages/validation-keys.ts
next-backend/src/messages/validation.ts
next-backend/src/tests
next-backend/src/tests/README.ts
```

Annexe C - Routes frontend principales

Route	Usage
/login	Connexion utilisateur
/admin/signup	Demande d'inscription administrateur

Route	Usage
/dashboard	Tableau de bord administrateur
/import/clients	Import clients Excel / CSV / JSON
/import/errors	Rapport d'erreurs d'import
/import/success	Rapport de succes d'import
/clients	Liste des clients
/clients/add	Ajout client
/clients/:id	Fiche detail client
/rentals	Liste des locations
/rentals/add	Creation location
/rentals/:id	Detail location
/payments	Liste des paiements mensuels
/payments/add	Ajout paiement
/payments/history	Historique des paiements
/payments/receipts	Recus et export PDF
/payments/deposit	Gestion des cautions
/subscription	Paiement abonnement administrateur
/documents	Gestion des documents
/documents/dossiers	Dossiers client

Route	Usage
/documents/contracts	Contrats signes
/archive/clients	Clients archives
/archive/blacklist	Clients blacklistes
/settings	Parametres administrateur / gouvernance
/work	Travaux et actions a faire
/danger/clients	Clients en risque
/pmt/admin	Portail Super Admin
/pmt/admin/entreprises	Gestion entreprises
/pmt/admin/admins	Gestion admins et demandes
/pmt/admin/stats	Statistiques globales
/pmt/admin/notifications	Notifications supervision
/pmt/admin/settings	Parametres Super Admin
/pmt/admin/logs	Logs et audit
/pmt/admin/monitoring/requests	Suivi des requetes
/pmt/admin/monitoring/performance	Suivi performance

Annexe D - Endpoints backend exposes

Endpoint
/api/admin_payments
/api/admin_payments/[id]
/api/admin_payments/status
/api/admin_requests
/api/admin_requests/[id]
/api/admins
/api/admins/[id]
/api/audit_logs
/api/audit_logs/[id]
/api/auth/login
/api/auth/logout
/api/auth/pending-check
/api/auth/session
/api/authContext
/api/authContext/clear-impersonation
/api/authContext/impersonate
/api/authContext/login

Endpoint

/api/authContext/logout

/api/authContext/rafraichir
/api/authContext/super-admin/second-auth

/api/authContext/super-admin/totp/activer

/api/authContext/super-admin/totp/initialiser

/api/authContext/super-admin/totp/statut

/api/blocked_ips

/api/blocked_ips/[id]

/api/clients

/api/clients/[id]

/api/cloudinary/open-url

/api/deposits

/api/deposits/[id]

/api/documentation

/api/documents

/api/documents/[id]

/api/entreprises

/api/entreprises/[id]

/api/import_runs

Endpoint

/api/import_runs/[id]

/api/locations

/api/locations/[id]

/api/notifications

/api/notifications/[id]

/api/notifications/clients/impayes

/api/payments

/api/payments/[id]

/api/sante

/api/securite/audits

/api/securite/super-admin/rapport-hebdo

/api/settings

/api/settings/[id]

/api/sign

/api/undo-actions

/api/undo-actions/[id]/rollback

/api/users

/api/users/[id]

/api/work_items

Endpoint

/api/work_items/[id]

Annexe E - Extraits exacts du code

FE-01 - Routage principal de l application React

Cet extrait prouve la separation des routes communes, administrateur et Super Admin. Il montre aussi l usage de MainLayout, des providers globaux, du badge offline et des pages metier.

Source : src/App.tsx:1-174

```
 1 | import { useEffect } from "react";
 2 | import { Toaster } from "@/components/ui/toaster";
 3 | import { TooltipProvider } from "@/components/ui/tooltip";
 4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
 5 | import { BrowserRouter, HashRouter, Routes, Route, Navigate } from "react-router-dom";
 6 | import { I18nProvider } from "@/lib/i18n";
 7 | import { AuthProvider } from "@/contexts/AuthContext";
 8 | import { DataProvider } from "@/stores/DataProvider";
 9 | import { ToastProvider } from "@/contexts/ToastContext";
10 | import { ToastContainer } from "@/components/ToastContainer";
11 | import { ErrorBoundary } from "@/components/ErrorBoundary";
12 | import { OfflineSyncBadge } from "@/components/OfflineSyncBadge";
13 | import { MainLayout } from "@/layouts/MainLayout";
14 | import Login from "./pages/common/Login";
15 | import Dashboard from "./pages/admin/Dashboard";
16 | import Settings from "./pages/admin/Settings";
17 | import PrivateAdminRoute from '@/components/PrivateAdminRoute';
18 | import ImportClients from "./pages/admin/ImportClients";
19 | import ImportErrors from "./pages/admin/ImportErrors";
20 | import ImportSuccess from "./pages/admin/ImportSuccess";
21 | import Clients from "./pages/admin/Clients";
22 | import AddClient from "./pages/admin/AddClient";
23 | import ClientDetail from "./pages/admin/ClientDetail";
24 | import Rentals from "./pages/admin/Rentals";
25 | import AddRental from "./pages/admin/AddRental";
26 | import EditRental from "./pages/admin/EditRental";
27 | import RentalDetail from "./pages/admin/RentalDetail";
28 | import Payments from "./pages/admin/Payments";
29 | import AddPayment from "./pages/admin/AddPayment";
```

```
  30 | import Deposits from "./pages/admin/Deposits";
  31 | import PaymentHistory from "./pages/admin/PaymentHistory";
  32 | import PaymentReceipts from "./pages/admin/PaymentReceipts";
  33 | import AdminSubscriptionPayments from "./pages/admin/AdminSubscriptionPayments";
  34 | import ClientDossier from "./pages/admin/ClientDossier";
  35 | import SignedContracts from "./pages/admin/SignedContracts";
  36 | import Documents from "./pages/admin/Documents";
  37 | import ArchivedClients from "./pages/admin/ArchivedClients";
  38 | import BlacklistedClients from "./pages/admin/BlacklistedClients";
  39 | import Archive from "./pages/admin/Archive";
  40 | import { Work } from "./pages/admin/Work";
  41 | import NotFound from "./pages/common/NotFound";
  42 | import SuperAdminPage from "./pages/super-admin/SuperAdminPage";
  43 | import EntreprisesPage from "./pages/super-admin/Entreprises";
  44 | import AdminsPage from "./pages/super-admin/admins/AdminsPage";
  45 | import SuperAdminStatsPage from "./pages/super-admin/SuperAdminStats";
  46 | import NotificationsPage from "./pages/super-admin/notifications/NotificationsPage";
  47 | import SuperAdminSettingsPage from
"./pages/super-admin/settings/SuperAdminSettingsPage";
  48 | import LogsPage from "./pages/super-admin/logs/LogsPage";
  49 | import RequestsPage from "./pages/super-admin/monitoring/RequestsPage";
  50 | import PerformancePage from "./pages/super-admin/monitoring/PerformancePage";
  51 | import AdminSignup from "./pages/common/AdminSignup";
  52 | import DangerClients from "./pages/admin/DangerClients";
  53 | import {
  54 |   OFFLINE_SYNC_QUEUE_UPDATED_EVENT,
  55 |   syncQueuedActions,
  56 | } from "@/infrastructure/syncQueue";
  57 |
  58 | const queryClient = new QueryClient();
  59 | const isElectronDesktop =
  60 |   typeof navigator !== "undefined" && /electron/i.test(navigator.userAgent);
  61 | const Router = isElectronDesktop ? HashRouter : BrowserRouter;
  62 |
  63 | const App = () => {
  64 |   useEffect(() => {
  65 |     let isSyncRunning = false
  66 |     const runSync = async () => {
  67 |       if (isSyncRunning) return
  68 |       if (typeof navigator !== "undefined" && navigator.onLine === false) return
  69 |       isSyncRunning = true
  70 |       try {
  71 |         await syncQueuedActions()
  72 |       } catch {
  73 |         // ignore sync errors, queue will retry on next online event
  74 |       } finally {
  75 |         isSyncRunning = false
  76 |       }
  77 |     }
  78 |
```

```
  79 |     const handleOnline = () => {
  80 |       void runSync()
  81 |     }
  82 |     const handleQueueUpdated = () => {
  83 |       if (typeof navigator !== "undefined" && navigator.onLine === false) return
  84 |       void runSync()
  85 |     }
  86 |     let intervalId: number | null = null
  87 |
  88 |     if (typeof window !== "undefined") {
  89 |       window.addEventListener("online", handleOnline)
  90 |       window.addEventListener(OFFLINE_SYNC_QUEUE_UPDATED_EVENT, handleQueueUpdated)
  91 |       intervalId = window.setInterval(() => {
  92 |         void runSync()
  93 |       }, 30_000)
  94 |     }
  95 |
  96 |     if (typeof navigator !== "undefined" && navigator.onLine) {
  97 |       void runSync()
  98 |     }
  99 |
 100 |     return () => {
 101 |       if (typeof window !== "undefined") {
 102 |         window.removeEventListener("online", handleOnline)
 103 |         window.removeEventListener(OFFLINE_SYNC_QUEUE_UPDATED_EVENT, handleQueueUpdated)
 104 |         if (intervalId !== null) {
 105 |           window.clearInterval(intervalId)
 106 |         }
 107 |       }
 108 |     }
 109 |   }, [])
 110 |
 111 |   return (
 112 |     <ErrorBoundary>
 113 |       <QueryClientProvider client={queryClient}>
 114 |         <I18nProvider>
 115 |           <AuthProvider>
 116 |             <DataProvider>
 117 |               <ToastProvider>
 118 |               <TooltipProvider>
 119 |                 <Toaster />
 120 |                 <ToastContainer />
 121 |                 <OfflineSyncBadge />
 122 |                 <Router future={{ v7_startTransition: true, v7_relativeSplatPath:
true }}>
 123 |                 <Routes>
 124 |                   <Route path="/login" element={<Login />} />
 125 |                   <Route path="/admin/signup" element={<AdminSignup />} />
 126 |                   <Route path="/pmt/admin" element={<SuperAdminPage />} />
 127 |                   <Route path="/pmt/admin/entreprises" element={<EntreprisesPage />} />
```

```
 128 |                    <Route path="/pmt/admin/admins" element={<AdminsPage />} />
 129 |                    <Route path="/pmt/admin/stats" element={<SuperAdminStatsPage />} />
 130 |                    <Route path="/pmt/admin/notifications" element={<NotificationsPage />}
/>
 131 |                    <Route path="/pmt/admin/settings" element={<SuperAdminSettingsPage />}
/>
 132 |                    <Route path="/pmt/admin/logs" element={<LogsPage />} />
 133 |                    <Route path="/pmt/admin/monitoring/requests" element={<RequestsPage
/>} />
 134 |                    <Route path="/pmt/admin/monitoring/performance"
element={<PerformancePage />} />
 135 |                    <Route path="/" element={<Navigate to="/login" replace />} />
 136 |                    <Route element={<MainLayout />}>
 137 |                      <Route path="/dashboard" element={<Dashboard />} />
 138 |                      <Route path="/import/clients" element={<ImportClients />} />
 139 |                      <Route path="/import/errors" element={<ImportErrors />} />
 140 |                      <Route path="/import/success" element={<ImportSuccess />} />
 141 |
 142 |                      {/* Clients Routes */}
 143 |                      <Route path="/clients" element={<Clients />} />
 144 |                      <Route path="/clients/add" element={<AddClient />} />
 145 |                      <Route path="/clients/:id" element={<ClientDetail />} />
 146 |                      <Route path="/clients/:id/edit" element={<AddClient />} />
 147 |                      <Route path="/clients/:id/add-rental" element={<AddRental />} />
 148 |
 149 |                      {/* Rentals Routes */}
 150 |                      <Route path="/rentals" element={<Rentals />} />
 151 |                      <Route path="/rentals/add" element={<AddRental />} />
 152 |                      <Route path="/rentals/add/:clientId" element={<AddRental />} />
 153 |                      <Route path="/rentals/:id" element={<RentalDetail />} />
 154 |                      <Route path="/rentals/:id/edit" element={<EditRental />} />
 155 |
 156 |                      {/* Payments Routes - Note: IDs can contain dashes like client-6-
rental-1 */}
 157 |                      <Route path="/payments" element={<Payments />} />
 158 |                      <Route path="/payments/add" element={<AddPayment />} />
 159 |                      <Route path="/payments/add/:clientId" element={<AddPayment />} />
 160 |                      <Route path="/payments/add/:clientId/:rentalId" element={<AddPayment
/>} />
 161 |                      <Route path="/payments/:rentalId/edit/:paymentId"
element={<AddPayment />} />
 162 |                      <Route path="/payments/edit/:paymentId" element={<AddPayment />} />
 163 |                      <Route path="/payments/:rentalId" element={<RentalDetail />} />
 164 |                      <Route path="/payments/deposit" element={<Deposits />} />
 165 |                      <Route path="/payments/deposit/:rentalId/edit" element={<Deposits
/>} />
 166 |                      <Route path="/payments/deposit/:rentalId" element={<Deposits />} />
 167 |                      <Route path="/payments/history" element={<PaymentHistory />} />
 168 |                      <Route path="/payments/receipts" element={<PaymentReceipts />} />
 169 |                      <Route path="/subscription" element={<AdminSubscriptionPayments
```

```
/>} />
 170 |
 171 |                     {/* Documents & Archive Routes */}
 172 |                     <Route path="/documents/dossiers" element={<ClientDossier />} />
 173 |                     <Route path="/documents/contracts" element={<SignedContracts />} />
 174 |                     <Route path="/documents/archive" element={<Documents />} />
```

FE-02 - Gestion de session cote frontend

Le contexte Auth restaure automatiquement la session, gere le login, la seconde authentification Super Admin et la deconnexion.

Source : src/contexts/AuthContext.tsx:1-170

```
   1 | import React, { createContext, useContext, useState, useCallback, ReactNode, useEffect,
useRef } from 'react';
   2 | import {
   3 |   clearImpersonation as clearImpersonationApi,
   4 |   getAuthContext,
   5 |   ImpersonationState,
   6 |   loginAuthContext,
   7 |   logoutAuthContext,
   8 |   setImpersonation as setImpersonationApi,
   9 |   verifySuperAdminSecondAuth as verifySuperAdminSecondAuthApi,
  10 | } from '@/services/api';
  11 | import {
  12 |   clearFailedLoginAttempts,
  13 |   getPlatformConfigSnapshot,
  14 |   refreshPlatformConfigFromServer,
  15 | } from '@/services/platformConfig';
  16 | import type { AdminFeaturePermissions } from '@/dto/frontend/responses';
  17 |
  18 | export type User = {
  19 |   id: string;
  20 |   username: string;
  21 |   name: string;
  22 |   email: string;
  23 |   role: string;
  24 |   status?: string;
  25 |   subscriptionBlocked?: boolean;
  26 |   subscriptionOverdueMonth?: string | null;
  27 |   subscriptionDueAt?: string | null;
  28 |   subscriptionRequiredMonth?: string | null;
  29 |   superAdminSecondAuthRequired?: boolean;
  30 |   permissions?: AdminFeaturePermissions;
  31 | };
```

```
  32 |
  33 | type AuthContextType = {
  34 |   user: User | null;
  35 |   isAuthenticated: boolean;
  36 |   isLoading: boolean;
  37 |   impersonation: ImpersonationState;
  38 |   login: (username: string, password: string) => Promise<boolean>;
  39 |   verifySuperAdminSecondAuth: (password: string) => Promise<boolean>;
  40 |   logout: () => void;
  41 |   setImpersonation: (payload: ImpersonationState) => Promise<void>;
  42 |   clearImpersonation: () => Promise<void>;
  43 | }
  44 |
  45 | const AuthContext = createContext<AuthContextType | undefined>(undefined);
  46 |
  47 | type AuthProviderProps = {
  48 |   children: ReactNode;
  49 | }
  50 |
  51 | function isPublicAuthRoute(pathname: string): boolean {
  52 |   const safePath = String(pathname || '').trim();
  53 |   return (
  54 |     safePath === '/login' ||
  55 |     safePath === '/login/' ||
  56 |     safePath === '/admin/signup' ||
  57 |     safePath === '/admin/signup/' ||
  58 |     safePath === '/signup' ||
  59 |     safePath === '/signup/'
  60 |   );
  61 | }
  62 |
  63 | export function AuthProvider({ children }: AuthProviderProps) {
  64 |   const [user, setUser] = useState<User | null>(null);
  65 |   const [isLoading, setIsLoading] = useState(true);
  66 |   const [impersonation, setImpersonationState] = useState<ImpersonationState>(null);
  67 |   const secondAuthProbeInFlightRef = useRef(false);
  68 |   const SESSION_LOGIN_AT_KEY = 'kya_session_login_at';
  69 |   const SESSION_LAST_ACTIVITY_KEY = 'kya_session_last_activity';
  70 |   const LAST_LOGIN_USERNAME_KEY = 'kya_last_login_username';
  71 |   const LAST_LOGIN_IDENTIFIER_KEY = 'kya_last_login_identifier';
  72 |
  73 |   // Restore user session from backend (persistent)
  74 |   useEffect(() => {
  75 |     let mounted = true;
  76 |     const loadSession = async () => {
  77 |       const isElectronDesktop =
  78 |         typeof navigator !== 'undefined' && /electron/i.test(String(navigator.userAgent
|| ''));
  79 |
  80 |       // Desktop behavior: always require explicit login when the app starts.
```

```
  81 |       if (isElectronDesktop) {
  82 |         if (!mounted) return;
  83 |         setUser(null);
  84 |         setImpersonationState(null);
  85 |         sessionStorage.removeItem(SESSION_LOGIN_AT_KEY);
  86 |         sessionStorage.removeItem(SESSION_LAST_ACTIVITY_KEY);
  87 |         setIsLoading(false);
  88 |         return;
  89 |       }
  90 |
  91 |       if (typeof window !== 'undefined' && isPublicAuthRoute(window.location.pathname))
{
  92 |         if (!mounted) return;
  93 |         setUser(null);
  94 |         setImpersonationState(null);
  95 |         setIsLoading(false);
  96 |         return;
  97 |       }
  98 |
  99 |       try {
 100 |         const ctx = await getAuthContext();
 101 |         if (!mounted) return;
 102 |         setUser(ctx.user as User | null);
 103 |         setImpersonationState(ctx.impersonation || null);
 104 |         if (ctx.user?.username) {
 105 |           localStorage.setItem(LAST_LOGIN_USERNAME_KEY, String(ctx.user.username));
 106 |         }
 107 |         const restoredIdentifier = String(ctx.user?.email || ctx.user?.username ||
'').trim();
 108 |         if (restoredIdentifier) {
 109 |           localStorage.setItem(LAST_LOGIN_IDENTIFIER_KEY, restoredIdentifier);
 110 |         }
 111 |         if (ctx.user?.id && !sessionStorage.getItem(SESSION_LOGIN_AT_KEY)) {
 112 |           sessionStorage.setItem(SESSION_LOGIN_AT_KEY, String(Date.now()));
 113 |         }
 114 |         if (ctx.user?.id) {
 115 |           sessionStorage.setItem(SESSION_LAST_ACTIVITY_KEY, String(Date.now()));
 116 |         }
 117 |       } catch {
 118 |         if (!mounted) return;
 119 |         setUser(null);
 120 |         setImpersonationState(null);
 121 |       } finally {
 122 |         if (mounted) setIsLoading(false);
 123 |       }
 124 |     };
 125 |     loadSession();
 126 |     return () => {
 127 |       mounted = false;
 128 |     };
```

```
 129 |   }, []);
 130 |
 131 |   const login = useCallback(async (username: string, password: string): Promise<boolean>
=> {
 132 |     const payload = await loginAuthContext(username, password);
 133 |     const userData = payload?.user;
 134 |     if (userData) {
 135 |       setUser(userData as User);
 136 |       setImpersonationState(null);
 137 |       setIsLoading(false);
 138 |       clearFailedLoginAttempts();
 139 |       const normalizedIdentifier = String(username || '').trim();
 140 |       const storedIdentifier = normalizedIdentifier || String(userData.email ||
userData.username || '').trim();
 141 |       if (storedIdentifier) {
 142 |         localStorage.setItem(LAST_LOGIN_IDENTIFIER_KEY, storedIdentifier);
 143 |       }
 144 |       localStorage.setItem(LAST_LOGIN_USERNAME_KEY, String(userData.username ||
storedIdentifier || ''));
 145 |       sessionStorage.setItem(SESSION_LOGIN_AT_KEY, String(Date.now()));
 146 |       sessionStorage.setItem(SESSION_LAST_ACTIVITY_KEY, String(Date.now()));
 147 |       return true;
 148 |     }
 149 |     return false;
 150 |   }, []);
 151 |
 152 |   const verifySuperAdminSecondAuth = useCallback(async (password: string):
Promise<boolean> => {
 153 |     const payload = await verifySuperAdminSecondAuthApi(password, user?.email ||
user?.username);
 154 |     const userData = payload?.user;
 155 |     if (userData) {
 156 |       setUser(userData as User);
 157 |       setIsLoading(false);
 158 |       const existingIdentifier = String(localStorage.getItem(LAST_LOGIN_IDENTIFIER_KEY)
|| '').trim();
 159 |       const storedIdentifier = existingIdentifier || String(userData.email ||
user?.email || userData.username || user?.username || '').trim();
 160 |       if (storedIdentifier) {
 161 |         localStorage.setItem(LAST_LOGIN_IDENTIFIER_KEY, storedIdentifier);
 162 |       }
 163 |       localStorage.setItem(LAST_LOGIN_USERNAME_KEY, String(userData.username ||
user?.username || ''));
 164 |       sessionStorage.setItem(SESSION_LAST_ACTIVITY_KEY, String(Date.now()));
 165 |       return true;
 166 |     }
 167 |     return false;
 168 |   }, [user?.email, user?.username]);
 169 |
 170 |   const logout = useCallback(() => {
```

FE-03 - Store global des donnees metier

Le store Zustand centralise les clients, les statistiques et les actions metier comme la creation client, les paiements, les documents et les synchronisations offline.

Source : src/stores/dataStore.ts:1-240

```
    1 | import { create } from 'zustand'
    2 | import { addDays, addMonths } from 'date-fns'
    3 | import type { Client, DashboardStats, MonthlyPayment, Rental } from '@/lib/types'
    4 | import type { ClientCreateDTO, ClientUpdateDTO } from '@/dto/backend/requests'
    5 | import { enqueueCreateClientAction, enqueueDeleteClientAction, enqueueUpdateClientAction
} from '@/infrastructure/syncQueue'
    6 | import {
    7 |   fetchClients as fetchClientsAPI,
    8 |   createClient,
    9 |   updateClient as updateClientAPI,
   10 |   deleteClient as deleteClientAPI,
   11 |   postPaymentRecord,
   12 |   postDepositPayment,
   13 |   deleteDocument as deleteDocumentAPI,
   14 |   updateMonthlyPayment as updateMonthlyPaymentAPI,
   15 |   uploadToCloudinary,
   16 | } from '@/services/api'
   17 | import { getAuthContext } from '@/services/api/auth.api'
   18 | import {
   19 |   normalizeEmailForCompare,
   20 |   normalizePhoneForCompare,
   21 |   validateEmail,
   22 |   validateName,
   23 |   validateSenegalNumber,
   24 | } from '@/validators/frontend'
   25 | import { calculateDashboardStats } from '@/services/data/stats'
   26 | import { generateId } from '@/services/data/normalizers'
   27 | import { serializeClientForApi } from '@/services/data/serialization'
   28 | import { transformClientDTO } from '@/services/data/transform'
   29 |
   30 | function isLikelyNetworkError(error: unknown): boolean {
   31 |   if (error instanceof TypeError) return true
   32 |   const message = String((error as { message?: string })?.message || error ||
'').toLowerCase()
   33 |   return (
   34 |     message.includes('networkerror') ||
   35 |     message.includes('failed to fetch') ||
   36 |     message.includes('network request failed')
   37 |   )
   38 | }
   39 |
   40 | function isBrowserOffline(): boolean {
```

```
  41 |   if (typeof navigator === 'undefined') return false
  42 |   return navigator.onLine === false
  43 | }
  44 |
  45 | export interface DataState {
  46 |   // State
  47 |   clients: Client[]
  48 |   stats: DashboardStats
  49 |   isLoading: boolean
  50 |   error: string | null
  51 |
  52 |   // Actions
  53 |   fetchClients: () => Promise<void>
  54 |   fetchStats: () => Promise<void>
  55 |   addClient: (
  56 |     client: Omit<Client, 'id' | 'createdAt' | 'rentals'> & {
  57 |       rental?: Omit<Rental, 'id' | 'clientId' | 'payments' | 'documents'>
  58 |     },
  59 |     options?: { refreshAfterSave?: boolean }
  60 |   ) => Promise<Client>
  61 |   updateClient: (id: string, data: Partial<Client>) => Promise<void>
  62 |   archiveClient: (id: string) => Promise<void>
  63 |   blacklistClient: (id: string) => Promise<void>
  64 |   getClient: (id: string) => Client | undefined
  65 |   addRental: (clientId: string, rental: Omit<Rental, 'id' | 'clientId' | 'payments' |
'documents'>) => Promise<void>
  66 |   deleteClient: (clientId: string) => Promise<void>
  67 |   addMonthlyPayment: (
  68 |     rentalId: string,
  69 |     paymentId: string,
  70 |     amount: number,
  71 |     options?: { date?: string; receiptNumber?: string; notes?: string }
  72 |   ) => Promise<void>
  73 |   editMonthlyPayment: (rentalId: string, paymentId: string, amount: number) =>
Promise<void>
  74 |   addDepositPayment: (
  75 |     rentalId: string,
  76 |     amount: number,
  77 |     options?: { date?: string; receiptNumber?: string; notes?: string }
  78 |   ) => Promise<void>
  79 |   addDocument: (
  80 |     clientId: string,
  81 |     rentalId: string,
  82 |     doc: { name: string; type: 'contract' | 'receipt' | 'other'; signed?: boolean;
file?: File | null }
  83 |   ) => Promise<void>
  84 |   deleteDocument: (clientId: string, rentalId: string, docId: string) => Promise<void>
  85 |   refreshStats: () => Promise<void>
  86 |   setError: (error: string | null) => void
  87 |   resetData: () => void
```

```
  88 | }
  89 |
  90 | /**
  91 |  * Zustand store for global data state management
  92 |  * Replaces Context API for better performance and simplicity
  93 |  * Manages clients, stats, and related operations
  94 |  */
  95 | export const useStore = <T>(selector: (state: DataState) => T) => useDataStore(selector)
  96 |
  97 | export const useDataStore = create<DataState>((set, get) => ({
  98 |   // Initial state
  99 |   clients: [],
 100 |   stats: { total: 0, active: 0, archived: 0, blacklisted: 0, totalDeposits: 0,
totalRents: 0 },
 101 |   isLoading: false,
 102 |   error: null,
 103 |
 104 |   // Fetch clients from API
 105 |   fetchClients: async () => {
 106 |     try {
 107 |       set({ isLoading: true, error: null })
 108 |       const [dtos, ctx] = await Promise.all([fetchClientsAPI(), getAuthContext()])
 109 |       const activeAdminId =
 110 |         ctx?.impersonation?.adminId ||
 111 |         (String(ctx?.user?.role || '').toUpperCase() === 'ADMIN' ? ctx?.user?.id || null
: null)
 112 |       const clients = dtos
 113 |         .map((d) => transformClientDTO(d))
 114 |         .filter((c) => c.firstName && c.lastName)
 115 |         .filter((c) => (activeAdminId ? c.adminId === activeAdminId : true))
 116 |       const stats = calculateDashboardStats(clients)
 117 |       set({ clients, stats, isLoading: false })
 118 |     } catch (error) {
 119 |       const message = error instanceof Error ? error.message : 'Failed to fetch clients'
 120 |       set({ error: message, isLoading: false })
 121 |       throw error
 122 |     }
 123 |   },
 124 |
 125 |   // Fetch stats from API (calculated from clients)
 126 |   fetchStats: async () => {
 127 |     try {
 128 |       const [dtos, ctx] = await Promise.all([fetchClientsAPI(), getAuthContext()])
 129 |       const activeAdminId =
 130 |         ctx?.impersonation?.adminId ||
 131 |         (String(ctx?.user?.role || '').toUpperCase() === 'ADMIN' ? ctx?.user?.id || null
: null)
 132 |       const clients = dtos
 133 |         .map((d) => transformClientDTO(d))
 134 |         .filter((c) => c.firstName && c.lastName)
```

```
 135 |         .filter((c) => (activeAdminId ? c.adminId === activeAdminId : true))
 136 |       const stats = calculateDashboardStats(clients)
 137 |       set({ stats, error: null })
 138 |     } catch (error) {
 139 |       const message = error instanceof Error ? error.message : 'Failed to fetch stats'
 140 |       set({ error: message })
 141 |       throw error
 142 |     }
 143 |   },
 144 |
 145 |   // Add new client
 146 |   addClient: async (clientData, options) => {
 147 |     try {
 148 |       set({ error: null })
 149 |       const shouldRefreshAfterSave = options?.refreshAfterSave !== false
 150 |       const ctx = await getAuthContext()
 151 |       const activeAdminId =
 152 |         ctx?.impersonation?.adminId ||
 153 |         (String(ctx?.user?.role || '').toUpperCase() === 'ADMIN' ? ctx?.user?.id ||
undefined : undefined)
 154 |       const scopedAdminId = String(activeAdminId || '').trim()
 155 |       const firstName = (clientData.firstName || '').trim()
 156 |       const lastName = (clientData.lastName || '').trim()
 157 |       const phone = (clientData.phone || '').trim()
 158 |       const email = (clientData.email || '').trim()
 159 |
 160 |       if (!firstName || !validateName(firstName)) throw new Error('Prénom invalide')
 161 |       if (!lastName || !validateName(lastName)) throw new Error('Nom invalide')
 162 |       if (!phone || !validateSenegalNumber(phone)) throw new Error('Numéro de téléphone
invalide')
 163 |       if (email && !validateEmail(email)) throw new Error('Email invalide')
 164 |
 165 |       const existingClientsForAdmin = scopedAdminId
 166 |         ? get().clients.filter((c) => String(c.adminId || '').trim() === scopedAdminId)
 167 |         : []
 168 |       const normalizedPhone = normalizePhoneForCompare(phone)
 169 |       const normalizedEmail = email ? normalizeEmailForCompare(email) : ''
 170 |       if (normalizedPhone) {
 171 |         const dup = existingClientsForAdmin.find((c) => normalizePhoneForCompare(c.phone
|| '') === normalizedPhone)
 172 |         if (dup) throw new Error('Client existe déjà avec ce numéro.')
 173 |       }
 174 |       if (normalizedEmail) {
 175 |         const dup = existingClientsForAdmin.find((c) => normalizeEmailForCompare(c.email
|| '') === normalizedEmail)
 176 |         if (dup) throw new Error('Client existe déjà avec cet email.')
 177 |       }
 178 |
 179 |       const clientId = generateId()
 180 |       const rentals: Rental[] = []
```

```
181 |       if (clientData.rental) {
182 |         const rentalId = generateId()
183 |         const startDate = clientData.rental.startDate
184 |         const monthlyRent = clientData.rental.monthlyRent
185 |
186 |         const initialPayment: MonthlyPayment = {
187 |           id: generateId(),
188 |           rentalId,
189 |           periodStart: startDate,
190 |           periodEnd: addDays(addMonths(startDate, 1), -1),
191 |           dueDate: addDays(addMonths(startDate, 1), 4),
192 |           amount: monthlyRent,
193 |           paidAmount: 0,
194 |           status: 'unpaid',
195 |           payments: [],
196 |         }
197 |
198 |         rentals.push({
199 |           id: rentalId,
200 |           clientId,
201 |           propertyType: clientData.rental.propertyType,
202 |           propertyName: clientData.rental.propertyName,
203 |           monthlyRent,
204 |           startDate: clientData.rental.startDate,
205 |           deposit: clientData.rental.deposit,
206 |           payments: [initialPayment],
207 |           documents: [],
208 |         })
209 |       }
210 |
211 |       const newClient: Client = {
212 |         id: clientId,
213 |         adminId: scopedAdminId || undefined,
214 |         firstName,
215 |         lastName,
216 |         phone,
217 |         email,
218 |         cni: clientData.cni,
219 |         status: clientData.status,
220 |         createdAt: new Date(),
221 |         rentals,
222 |       }
223 |
224 |       const payload = serializeClientForApi({ ...newClient, createdAt:
newClient.createdAt })
225 |       const createPayload = payload as ClientCreateDTO
226 |
227 |       const applyClientOptimistically = () => {
228 |         set((state) => {
229 |           const clients = [...state.clients, newClient]
```

```
230 |           return {
231 |             clients,
232 |             stats: calculateDashboardStats(clients),
233 |             error: null,
234 |           }
235 |         })
236 |       }
237 |
238 |       if (isBrowserOffline()) {
239 |         await enqueueCreateClientAction(createPayload)
240 |         applyClientOptimistically()
```

FE-04 - Flux complet d import clients

Ce bloc montre l analyse, le calcul des erreurs, le parcours des lignes valides, l enregistrement des import runs et la navigation vers les pages succes / erreurs.

Source : src/pages/admin/imports/ImportClientsPage.tsx:277-405

```
277 |   const saveErrorsToServer = async (nextErrors: ClientImportError[]) => {
278 |     if (isSavingErrors) return
279 |     setIsSavingErrors(true)
280 |     try {
281 |       const importRun = await createImportRun({
282 |         createdAt: new Date().toISOString(),
283 |         adminId: activeAdminId || undefined,
284 |         fileName,
285 |         totalRows: rows.length,
286 |         inserted: [],
287 |         errors: nextErrors.map((err) => ({
288 |           rowNumber: err.rowNumber,
289 |           errors: err.errors,
290 |           parsed: err.parsed,
291 |         })),
292 |         ignored: false,
293 |         readSuccess: true,
294 |         readErrors: false,
295 |       })
296 |       navigate('/import/errors', {
297 |         state: {
298 |           importRunId: importRun.id,
299 |           source: 'save-errors',
300 |         },
301 |       })
302 |     } catch (error) {
303 |       const message = error instanceof Error ? error.message : "Impossible d'enregistrer
```

```
les erreurs"
 304 |       toast({ title: 'Erreur', description: message, variant: 'destructive' })
 305 |     } finally {
 306 |       setIsSavingErrors(false)
 307 |     }
 308 |   }
 309 |
 310 |   const handleImport = async () => {
 311 |     if (!hasData || isImporting) return
 312 |     setIsImporting(true)
 313 |     setImportProgress({ processed: 0, total: 0 })
 314 |
 315 |     try {
 316 |       const nextErrors = collectErrors()
 317 |       if (!nextErrors) return
 318 |       setErrors(nextErrors)
 319 |       setShowFix(nextErrors.length > 0)
 320 |
 321 |       const invalidRowIndexes = new Set(nextErrors.map((error) => error.rowIndex))
 322 |       const rowsToImport = rows
 323 |         .map((row, idx) => ({ row, idx }))
 324 |         .filter(({ row, idx }) => {
 325 |           const hasContent = row.some(
 326 |             (cell) => cell !== null && cell !== undefined && String(cell).trim() !== ''
 327 |           )
 328 |           return hasContent && !invalidRowIndexes.has(idx)
 329 |         })
 330 |
 331 |       setImportProgress({ processed: 0, total: rowsToImport.length })
 332 |
 333 |       const inserted: Array<{ id: string; firstName: string; lastName: string; phone:
string; email?: string }> = []
 334 |
 335 |       for (let position = 0; position < rowsToImport.length; position++) {
 336 |         const { row, idx } = rowsToImport[position]
 337 |         setImportProgress({ processed: position + 1, total: rowsToImport.length })
 338 |         const parsed = buildRow(row, mapping, overrides[idx])
 339 |
 340 |         try {
 341 |           const result = await withImportTimeout(
 342 |             addClient(buildImportClientPayload(parsed), { refreshAfterSave: false }),
 343 |             `Ligne ${idx + 2}`
 344 |           )
 345 |
 346 |           inserted.push({
 347 |             id: result.id,
 348 |             firstName: result.firstName || '',
 349 |             lastName: result.lastName || '',
 350 |             phone: result.phone || '',
 351 |             email: result.email,
```

```
352 |           })
353 |         } catch (err) {
354 |           nextErrors.push({
355 |             rowIndex: idx,
356 |             rowNumber: idx + 2,
357 |             errors: [formatBackendError(err, parsed, ownerByPhone, ownerByEmail)],
358 |             raw: row,
359 |             parsed,
360 |           })
361 |         }
362 |       }
363 |
364 |       if (inserted.length > 0) {
365 |         await withImportTimeout(fetchClients(), 'Rafraîchissement des clients importés')
366 |       }
367 |
368 |       const importRun = await withImportTimeout(createImportRun({
369 |         createdAt: new Date().toISOString(),
370 |         adminId: activeAdminId || undefined,
371 |         fileName,
372 |         totalRows: rows.length,
373 |         inserted,
374 |         errors: nextErrors.map((err) => ({
375 |           rowNumber: err.rowNumber,
376 |           errors: err.errors,
377 |           parsed: err.parsed,
378 |         })),
379 |         ignored: false,
380 |         readSuccess: inserted.length === 0,
381 |         readErrors: nextErrors.length === 0,
382 |       }), "Enregistrement du journal d'import")
383 |
384 |       toast({
385 |         title: 'Import terminé',
386 |         description: rowsToImport.length === 0
387 |           ? "Aucune ligne valide à importer. Corrigez les erreurs puis réessayez."
388 |           : nextErrors.length > 0
389 |           ? `${inserted.length} client(s) inséré(s), ${nextErrors.length} erreur(s)`
390 |           : `${inserted.length} client(s) importé(s)`,
391 |       })
392 |
393 |       navigate(nextErrors.length > 0 ? '/import/errors' : '/import/success', {
394 |         state: {
395 |           importRunId: importRun.id,
396 |           source: 'import-clients',
397 |         },
398 |       })
399 |     } catch (error) {
400 |       const message = error instanceof Error ? error.message : "Échec de l'import."
401 |       toast({ title: 'Erreur', description: message, variant: 'destructive' })
```

```
402 |     } finally {
403 |       setImportProgress({ processed: 0, total: 0 })
404 |       setIsImporting(false)
405 |     }
```

FE-05 - Configuration runtime de l API

Le frontend peut changer son URL API au runtime, ce qui est utile pour la version web sur Vercel et la version desktop Electron.

Source : src/services/runtimeConfig.ts:1-180

```
  1 | type RuntimeConfigPayload = {
  2 |   apiBaseUrl?: string
  3 |   cloudinarySignUrl?: string
  4 | }
  5 |
  6 | type RuntimeConfigResponse = {
  7 |   success?: boolean
  8 |   error?: string
  9 |   config?: RuntimeConfigPayload
 10 |   userPath?: string
 11 |   portablePath?: string
 12 |   source?: string
 13 |   writtenPath?: string
 14 | }
 15 |
 16 | type RuntimeConfigState = {
 17 |   apiBaseUrl: string
 18 |   cloudinarySignUrl: string
 19 |   source: 'build-env' | 'local-storage' | 'user-file' | 'portable-file' | 'default'
 20 |   loaded: boolean
 21 |   userPath: string | null
 22 |   portablePath: string | null
 23 |   writtenPath: string | null
 24 | }
 25 |
 26 | type ElectronRuntimeApi = {
 27 |   getRuntimeConfig?: () => Promise<RuntimeConfigResponse>
 28 |   setRuntimeConfig?: (payload: RuntimeConfigPayload) => Promise<RuntimeConfigResponse>
 29 | }
 30 |
 31 | const URL_PROTOCOLS = new Set(['http:', 'https:'])
 32 | const LOCAL_HOSTS = new Set(['localhost', '127.0.0.1', '0.0.0.0', '::1'])
 33 | const API_PATH_SUFFIX = '/api'
 34 |
```

```
  35 | const RUNTIME_API_BASE_KEY = 'kya_runtime_api_base'
  36 | const RUNTIME_SIGN_URL_KEY = 'kya_runtime_sign_url'
  37 |
  38 | function isPrivateIpv4Host(hostname: string): boolean {
  39 |   const match = String(hostname || '').trim().match(/^(\d{1,3})\.(\d{1,3})\.(\d{1,3})\.
(\d{1,3})$/)
  40 |   if (!match) return false
  41 |
  42 |   const octets = match.slice(1).map((value) => Number(value))
  43 |   if (octets.some((value) => !Number.isFinite(value) || value < 0 || value > 255))
return false
  44 |
  45 |   const [first, second] = octets
  46 |   if (first === 10) return true
  47 |   if (first === 172 && second >= 16 && second <= 31) return true
  48 |   if (first === 192 && second === 168) return true
  49 |   if (first === 169 && second === 254) return true
  50 |   return false
  51 | }
  52 |
  53 | function isLocalHost(hostname: string): boolean {
  54 |   const safeHostname = String(hostname || '').trim().toLowerCase()
  55 |   if (!safeHostname) return false
  56 |   if (LOCAL_HOSTS.has(safeHostname)) return true
  57 |   if (safeHostname.endsWith('.local')) return true
  58 |   return isPrivateIpv4Host(safeHostname)
  59 | }
  60 |
  61 | function isPublicWebContext(): boolean {
  62 |   if (typeof window === 'undefined') return false
  63 |   const currentHost = String(window.location?.hostname || '').trim().toLowerCase()
  64 |   return Boolean(currentHost) && !isLocalHost(currentHost)
  65 | }
  66 |
  67 | function shouldRejectLocalRuntimeUrl(value: string): boolean {
  68 |   const raw = String(value || '').trim()
  69 |   if (!raw || !isPublicWebContext()) return false
  70 |   try {
  71 |     const parsed = new URL(raw)
  72 |     return isLocalHost(parsed.hostname)
  73 |   } catch {
  74 |     return false
  75 |   }
  76 | }
  77 |
  78 | function normalizeRuntimeUrl(value: string, label: string, stripTrailingSlash = false):
string {
  79 |   const raw = String(value || '').trim()
  80 |   if (!raw) return ''
  81 |
```

```
  82 |   let parsed: URL
  83 |   try {
  84 |     parsed = new URL(raw)
  85 |   } catch {
  86 |     throw new Error(`${label} invalide.`)
  87 |   }
  88 |
  89 |   if (!URL_PROTOCOLS.has(parsed.protocol)) {
  90 |     throw new Error(`${label} doit utiliser http(s).`)
  91 |   }
  92 |
  93 |   if (parsed.username || parsed.password) {
  94 |     throw new Error(`${label} ne doit pas contenir d'identifiants.`)
  95 |   }
  96 |
  97 |   if (parsed.protocol === 'http:' && !isLocalHost(parsed.hostname)) {
  98 |     throw new Error(`${label} doit être en https hors réseau local.`)
  99 |   }
 100 |
 101 |   const serialized = parsed.toString()
 102 |   if (stripTrailingSlash) return serialized.replace(/\/+$/, '')
 103 |   return serialized
 104 | }
 105 |
 106 | export function validateRuntimeApiBaseUrl(value: string): string {
 107 |   const normalized = normalizeRuntimeUrl(value, "L'URL de l'API backend", true)
 108 |   const parsed = new URL(normalized)
 109 |
 110 |   const normalizedPath = String(parsed.pathname || '').replace(/\/+$/, '')
 111 |   if (!normalizedPath || normalizedPath === '/') {
 112 |     parsed.pathname = API_PATH_SUFFIX
 113 |   } else if (normalizedPath === API_PATH_SUFFIX) {
 114 |     parsed.pathname = API_PATH_SUFFIX
 115 |   } else {
 116 |     throw new Error("L'URL de l'API backend doit pointer vers la racine du serveur ou
/api.")
 117 |   }
 118 |
 119 |   parsed.search = ''
 120 |   parsed.hash = ''
 121 |   return parsed.toString().replace(/\/+$/, '')
 122 | }
 123 |
 124 | export function validateRuntimeSignUrl(value: string): string {
 125 |   return normalizeRuntimeUrl(value, "L'URL de signature Cloudinary", false)
 126 | }
 127 |
 128 | const normalizeApiBaseUrl = (value: string): string => validateRuntimeApiBaseUrl(value)
 129 | const normalizeSignUrl = (value: string): string => validateRuntimeSignUrl(value)
 130 |
```

```
131 | const normalizeApiBaseUrlSafe = (value: string): string => {
132 |   try {
133 |     return normalizeApiBaseUrl(value)
134 |   } catch {
135 |     return ''
136 |   }
137 | }
138 |
139 | const normalizeSignUrlSafe = (value: string): string => {
140 |   try {
141 |     return normalizeSignUrl(value)
142 |   } catch {
143 |     return ''
144 |   }
145 | }
146 |
147 | const FALLBACK_API_ORIGIN = 'https://bakend-next-saas-gestion-client.onrender.com'
148 | const ENV_API_URL = String(import.meta.env.VITE_API_URL || '').trim()
149 | const getBrowserSameOriginApiBase = (): string => {
150 |   if (typeof window === 'undefined') return ''
151 |   const origin = String(window.location?.origin || '').trim()
152 |   if (!origin) return ''
153 |   try {
154 |     const parsed = new URL(origin)
155 |     if (!URL_PROTOCOLS.has(parsed.protocol)) return ''
156 |     parsed.pathname = API_PATH_SUFFIX
157 |     parsed.search = ''
158 |     parsed.hash = ''
159 |     return parsed.toString().replace(/\/+$/, '')
160 |   } catch {
161 |     return ''
162 |   }
163 | }
164 |
165 | const DEFAULT_API_BASE = (() => {
166 |   const browserSameOriginApi = getBrowserSameOriginApiBase()
167 |   // In browser contexts, always prefer same-origin `/api` to avoid CORS dependency.
168 |   if (browserSameOriginApi) return browserSameOriginApi
169 |
170 |   const normalizedEnvApi = normalizeApiBaseUrlSafe(ENV_API_URL)
171 |   if (normalizedEnvApi) return normalizedEnvApi
172 |
173 |   return normalizeApiBaseUrlSafe(FALLBACK_API_ORIGIN) || `${FALLBACK_API_ORIGIN}$
{API_PATH_SUFFIX}`
174 | })()
175 | const DEFAULT_CLOUDINARY_SIGN_URL = normalizeSignUrlSafe(
176 |   String(import.meta.env.VITE_CLOUDINARY_SIGN_URL || `${DEFAULT_API_BASE}/sign`)
177 | )
178 |
179 | const runtimeState: RuntimeConfigState = {
```

```
  180 |   apiBaseUrl: DEFAULT_API_BASE,
```

BE-01 - Conteneur de dependances backend

Le backend est organise autour d un conteneur qui instancie la configuration, la securite, Prisma, les services applicatifs, les DAO, les ecouteurs de notifications et les controleurs.

Source : next-backend/src/coeur/conteneur/ConteneurDependances.ts:1-260

```
    1 | import { ServiceSante } from '@/src/application/services/sante/ServiceSante'
    2 | import { ControleurSante } from '@/src/controleurs/ControleurSante'
    3 | import { AdaptateurPrisma } from '@/src/infrastructure/base_de_donnees/AdaptateurPrisma'
    4 | import { ClientPrisma } from '@/src/infrastructure/base_de_donnees/ClientPrisma'
    5 | import { ValidateurZod } from '@/src/infrastructure/validateurs/ValidateurZod'
    6 | import { ConfigurationApplication } from
'@/src/coeur/configuration/ConfigurationApplication'
    7 | import { ConfigurationSecurite } from '@/src/coeur/configuration/ConfigurationSecurite'
    8 | import { ServiceHachageMotDePasseArgon2 } from
'@/src/infrastructure/securite/ServiceHachageMotDePasseArgon2'
    9 | import { ServiceJetonAccesJwt } from
'@/src/infrastructure/securite/ServiceJetonAccesJwt'
   10 | import { ServiceTotp } from '@/src/infrastructure/securite/ServiceTotp'
   11 | import { ServiceChiffrementSymetrique } from
'@/src/infrastructure/securite/ServiceChiffrementSymetrique'
   12 | import { ServiceAuditSecurite } from
'@/src/infrastructure/securite/ServiceAuditSecurite'
   13 | import { ServiceAuditSecuriteMemoire } from
'@/src/infrastructure/securite/ServiceAuditSecuriteMemoire'
   14 | import { ServiceAlerteSuperAdminWebhook } from
'@/src/infrastructure/alertes/ServiceAlerteSuperAdminWebhook'
   15 | import { ServiceEmailBrevo } from '@/src/infrastructure/alertes/ServiceEmailBrevo'
   16 | import { ServiceEvenementsNotification } from
'@/src/infrastructure/alertes/ServiceEvenementsNotification'
   17 | import { ServiceEcouteurEvenementsNotificationBrevo } from
'@/src/infrastructure/alertes/ServiceEcouteurEvenementsNotificationBrevo'
   18 | import { ServiceEcouteurEvenementsNotificationWebhook } from
'@/src/infrastructure/alertes/ServiceEcouteurEvenementsNotificationWebhook'
   19 | import { ServiceAuthentification } from
'@/src/application/services/authentification/ServiceAuthentification'
   20 | import { ServiceContexteAuthentification } from
'@/src/application/services/authentification/ServiceContexteAuthentification'
   21 | import { ServiceSessionAuthentification } from
'@/src/application/services/authentification/ServiceSessionAuthentification'
   22 | import { ServiceSecuriteSessionAuthentification } from
'@/src/application/services/authentification/ServiceSecuriteSessionAuthentification'
   23 | import { ServiceTotpSuperAdminAuthentification } from
```

```
'@/src/application/services/authentification/ServiceTotpSuperAdminAuthentification'
  24 | import { ServiceAutorisationAuthentification } from
'@/src/application/services/authentification/ServiceAutorisationAuthentification'
  25 | import { ServiceAuditAuthentification } from
'@/src/application/services/authentification/ServiceAuditAuthentification'
  26 | import { ServiceImpersonationAuthentification } from
'@/src/application/services/authentification/ServiceImpersonationAuthentification'
  27 | import { ValidateurAuthentification } from
'@/src/application/validateurs/ValidateurAuthentification'
  28 | import { ControleurAuthContext } from '@/src/controleurs/ControleurAuthContext'
  29 | import { ServiceCookiesAuthentification } from
'@/src/infrastructure/securite/ServiceCookiesAuthentification'
  30 | import { ServiceProtectionCsrf } from
'@/src/infrastructure/securite/ServiceProtectionCsrf'
  31 | import { AdaptateurRequeteSecurite } from
'@/src/infrastructure/securite/AdaptateurRequeteSecurite'
  32 | import { ServiceEntetesSecuriteHttp } from
'@/src/infrastructure/securite/ServiceEntetesSecuriteHttp'
  33 | import { ServiceCorsStrict } from '@/src/infrastructure/securite/ServiceCorsStrict'
  34 | import { UtilitairesSecurite } from '@/src/infrastructure/securite/UtilitairesSecurite'
  35 | import { ReponseHttp } from '@/src/infrastructure/http/ReponseHttp'
  36 | import { ContexteRequeteHttp } from '@/src/infrastructure/http/ContexteRequeteHttp'
  37 | import { ValidateurAuthentificationZod } from
'@/src/infrastructure/validateurs/ValidateurAuthentificationZod'
  38 | import {
  39 |   MappeurUtilisateurAuthentification,
  40 | } from '@/src/application/mappers'
  41 | import { DaoAuthentificationPrisma } from
'@/src/infrastructure/dao/prisma/authentification/DaoAuthentificationPrisma'
  42 | import { DaoAuthentificationMemoire } from
'@/src/infrastructure/dao/memoire/authentification/DaoAuthentificationMemoire'
  43 | import { InterfaceDaoAuthentification } from
'@/src/domaine/interfaces/dao/authentification/InterfaceDaoAuthentification'
  44 | import { InterfaceRepositoryAuthentification } from
'@/src/domaine/interfaces/repository/InterfaceRepositoryAuthentification'
  45 | import { InterfaceServiceHachageMotDePasse } from
'@/src/coeur/interfaces/InterfaceServiceHachageMotDePasse'
  46 | import { InterfaceServiceJetonAcces } from
'@/src/coeur/interfaces/InterfaceServiceJetonAcces'
  47 | import { InterfaceServiceTotp } from '@/src/coeur/interfaces/InterfaceServiceTotp'
  48 | import { InterfaceServiceChiffrement } from
'@/src/coeur/interfaces/InterfaceServiceChiffrement'
  49 | import { InterfaceServiceAuditSecurite } from
'@/src/coeur/interfaces/InterfaceServiceAuditSecurite'
  50 | import {
  51 |   TypeDestinataireNotification,
  52 |   TypeRoleDestinataireNotification,
  53 | } from '@/src/coeur/interfaces/InterfaceNotification'
  54 | import { InterfaceUtilitairesSecurite } from
'@/src/coeur/interfaces/InterfaceUtilitairesSecurite'
```

```
  55 | import { RepositoryAuthentificationPrisma } from
'@/src/infrastructure/repositories/prisma/RepositoryAuthentificationPrisma'
  56 | import { RepositoryAuthentificationMemoire } from
'@/src/infrastructure/repositories/memoire/RepositoryAuthentificationMemoire'
  57 | import { FabriqueSessionAuthentification } from
'@/src/application/fabriques/FabriqueSessionAuthentification'
  58 | import { FabriqueJetonRefresh } from '@/src/application/fabriques/FabriqueJetonRefresh'
  59 | import { ServiceAdministrationAdmin } from
'@/src/application/services/administration/ServiceAdministrationAdmin'
  60 | import { ControleurAdministrationAdmin } from
'@/src/controleurs/ControleurAdministrationAdmin'
  61 | import {
  62 |   DaoPaiementAbonnementAdminMemoire,
  63 |   DaoStatutAbonnementAdminMemoire,
  64 | } from '@/src/infrastructure/dao/memoire/administration'
  65 | import {
  66 |   DaoPaiementAbonnementAdminPrisma,
  67 |   DaoStatutAbonnementAdminPrisma,
  68 | } from '@/src/infrastructure/dao/prisma/administration'
  69 | import {
  70 |   DaoClientMemoire,
  71 |   DaoDocumentMemoire,
  72 |   DaoPaiementCautionMemoire,
  73 |   DaoTransactionPaiementMemoire,
  74 | } from '@/src/infrastructure/dao/memoire/locations'
  75 | import {
  76 |   DaoClientPrisma,
  77 |   DaoDocumentPrisma,
  78 |   DaoPaiementCautionPrisma,
  79 |   DaoTransactionPaiementPrisma,
  80 | } from '@/src/infrastructure/dao/prisma/locations'
  81 | import {
  82 |   DaoExecutionImportMemoire,
  83 |   DaoIpBloqueeMemoire,
  84 |   DaoItemTravailMemoire,
  85 |   DaoJournalAuditMemoire,
  86 |   DaoNotificationMemoire,
  87 | } from '@/src/infrastructure/dao/memoire/systeme'
  88 | import {
  89 |   DaoExecutionImportPrisma,
  90 |   DaoIpBloqueePrisma,
  91 |   DaoItemTravailPrisma,
  92 |   DaoJournalAuditPrisma,
  93 |   DaoNotificationPrisma,
  94 |   DaoParametreAdminPrisma,
  95 | } from '@/src/infrastructure/dao/prisma/systeme'
  96 | import { creerServiceAdministrationAdminSupervision } from
'@/src/coeur/conteneur/FabriqueServiceAdministrationAdminSupervision'
  97 | import { lirePolitiquePlateforme } from '@/src/infrastructure/http/politiquePlateforme'
  98 |
```

```
  99 | class ConteneurDependances {
 100 |   public configurationApplication = new ConfigurationApplication()
 101 |   public configurationSecurite = new ConfigurationSecurite()
 102 |   public utiliseMemoire =
 103 |     this.configurationApplication.driverPersistanceAuthentification() === 'memoire'
 104 |   public prisma = ClientPrisma.obtenirInstance()
 105 |   public clientBaseDeDonnees = new AdaptateurPrisma()
 106 |   public validateurEntree = new ValidateurZod()
 107 |   public validateurAuthentificationInfrastructure = new ValidateurAuthentificationZod()
 108 |   public validateurAuthentification = new ValidateurAuthentification(
 109 |     this.validateurAuthentificationInfrastructure
 110 |   )
 111 |   public utilitairesSecurite: InterfaceUtilitairesSecurite = new UtilitairesSecurite()
 112 |   public reponseHttp = new ReponseHttp()
 113 |   public contexteRequeteHttp = new ContexteRequeteHttp(this.utilitairesSecurite)
 114 |   public mappeurUtilisateurAuthentification = new MappeurUtilisateurAuthentification(
 115 |     this.configurationSecurite
 116 |   )
 117 |   public serviceHachageMotDePasse: InterfaceServiceHachageMotDePasse = new
ServiceHachageMotDePasseArgon2()
 118 |   public serviceJetonAcces: InterfaceServiceJetonAcces = new ServiceJetonAccesJwt(
 119 |     this.configurationSecurite.cleJwt(),
 120 |     this.configurationSecurite.dureeJetonAccesSecondes()
 121 |   )
 122 |   public serviceTotp: InterfaceServiceTotp = new ServiceTotp()
 123 |   public serviceChiffrement: InterfaceServiceChiffrement = new
ServiceChiffrementSymetrique(
 124 |     this.configurationSecurite.cleChiffrementTotp()
 125 |   )
 126 |   public daoAuthentificationPrisma: InterfaceDaoAuthentification = new
DaoAuthentificationPrisma(this.prisma)
 127 |   public daoAuthentificationMemoire = new DaoAuthentificationMemoire()
 128 |   public serviceAlerteSuperAdminWebhook = new ServiceAlerteSuperAdminWebhook(
 129 |     this.configurationSecurite.urlWebhookAlertesSuperAdmin()
 130 |   )
 131 |   public serviceNotificationBrevo = new ServiceEmailBrevo({
 132 |     cleApi: this.configurationSecurite.brevoCleApi(),
 133 |     expediteurEmail: this.configurationSecurite.brevoExpediteurEmail(),
 134 |     expediteurNom: this.configurationSecurite.brevoExpediteurNom(),
 135 |   })
 136 |   public serviceEvenementsNotification = new ServiceEvenementsNotification()
 137 |   public serviceEcouteurEvenementsNotificationBrevo =
 138 |     new ServiceEcouteurEvenementsNotificationBrevo({
 139 |       serviceNotification: this.serviceNotificationBrevo,
 140 |       destinatairesParRole: this.construireDestinatairesNotificationParRole(),
 141 |     })
 142 |   public serviceEcouteurEvenementsNotificationWebhook =
 143 |     new
ServiceEcouteurEvenementsNotificationWebhook(this.serviceAlerteSuperAdminWebhook)
 144 |   public serviceAuditSecuritePrisma: InterfaceServiceAuditSecurite = new
```

```
ServiceAuditSecurite(
 145 |     this.prisma,
 146 |     this.configurationSecurite.urlWebhookAlertes(),
 147 |     {
 148 |       apiToken: this.configurationSecurite.whatsappCloudApiToken(),
 149 |       phoneNumberId: this.configurationSecurite.whatsappCloudPhoneNumberId(),
 150 |       destination: this.configurationSecurite.whatsappCloudDestination(),
 151 |       apiVersion: this.configurationSecurite.whatsappCloudApiVersion(),
 152 |     },
 153 |     this.serviceAlerteSuperAdminWebhook
 154 |   )
 155 |   public serviceAuditSecuriteMemoire: InterfaceServiceAuditSecurite =
 156 |     new ServiceAuditSecuriteMemoire(this.daoAuthentificationMemoire)
 157 |   public serviceAuditSecurite: InterfaceServiceAuditSecurite =
 158 |     this.utiliseMemoire ? this.serviceAuditSecuriteMemoire :
this.serviceAuditSecuritePrisma
 159 |   public serviceCookiesAuthentification = new ServiceCookiesAuthentification({
 160 |     modeSecurise: this.configurationSecurite.modeCookieSecurise(),
 161 |     sameSite: this.configurationSecurite.modeSameSiteCookies(),
 162 |   })
 163 |   public serviceProtectionCsrf = new ServiceProtectionCsrf()
 164 |   public adaptateurRequeteSecurite = new
AdaptateurRequeteSecurite(this.serviceProtectionCsrf)
 165 |   public serviceEntetesSecuriteHttp = new ServiceEntetesSecuriteHttp()
 166 |   public serviceCorsStrict = new
ServiceCorsStrict(this.configurationSecurite.originesCorsAutorisees())
 167 |   public repositoryAuthentificationPrisma: InterfaceRepositoryAuthentification =
 168 |     new RepositoryAuthentificationPrisma(this.daoAuthentificationPrisma)
 169 |   public repositoryAuthentificationMemoire: InterfaceRepositoryAuthentification =
 170 |     new RepositoryAuthentificationMemoire(this.daoAuthentificationMemoire)
 171 |   public repositoryAuthentification: InterfaceRepositoryAuthentification =
 172 |     this.utiliseMemoire ? this.repositoryAuthentificationMemoire :
this.repositoryAuthentificationPrisma
 173 |   public fabriqueSessionAuthentification = new FabriqueSessionAuthentification()
 174 |   public fabriqueJetonRefresh = new FabriqueJetonRefresh()
 175 |   public serviceContexteAuthentification = new ServiceContexteAuthentification(
 176 |     this.repositoryAuthentification,
 177 |     this.serviceJetonAcces,
 178 |     this.mappeurUtilisateurAuthentification
 179 |   )
 180 |   public serviceSecuriteSessionAuthentification = new
ServiceSecuriteSessionAuthentification(
 181 |     this.repositoryAuthentification,
 182 |     this.configurationSecurite,
 183 |     this.serviceAuditSecurite,
 184 |     async () => {
 185 |       const politique = await lirePolitiquePlateforme(this.prisma)
 186 |       return {
 187 |         maxFailedLogins: politique.sessionSecurity.maxFailedLogins,
 188 |         lockoutMinutes: politique.sessionSecurity.lockoutMinutes,
```

```
 189 |       }
 190 |     }
 191 |   )
 192 |   public serviceSessionAuthentification = new ServiceSessionAuthentification(
 193 |     this.repositoryAuthentification,
 194 |     this.serviceHachageMotDePasse,
 195 |     this.serviceJetonAcces,
 196 |     this.serviceAuditSecurite,
 197 |     this.configurationSecurite,
 198 |     this.utilitairesSecurite,
 199 |     this.fabriqueSessionAuthentification,
 200 |     this.fabriqueJetonRefresh,
 201 |     this.mappeurUtilisateurAuthentification,
 202 |     this.serviceSecuriteSessionAuthentification,
 203 |     async () => {
 204 |       const politique = await lirePolitiquePlateforme(this.prisma)
 205 |       return {
 206 |         sessionDurationMinutes: politique.sessionSecurity.sessionDurationMinutes,
 207 |         inactivityTimeoutMinutes: politique.sessionSecurity.inactivityTimeoutMinutes,
 208 |       }
 209 |     }
 210 |   )
 211 |   public serviceTotpSuperAdminAuthentification = new
ServiceTotpSuperAdminAuthentification(
 212 |     this.serviceContexteAuthentification,
 213 |     this.serviceSecuriteSessionAuthentification,
 214 |     this.repositoryAuthentification,
 215 |     this.serviceTotp,
 216 |     this.serviceChiffrement,
 217 |     this.serviceAuditSecurite,
 218 |     this.serviceHachageMotDePasse,
 219 |     this.configurationApplication
 220 |   )
 221 |   public serviceAutorisationAuthentification = new
ServiceAutorisationAuthentification(this.serviceContexteAuthentification)
 222 |   public serviceAuditAuthentification = new
ServiceAuditAuthentification(this.repositoryAuthentification,
this.serviceAutorisationAuthentification)
 223 |   public serviceImpersonationAuthentification = new
ServiceImpersonationAuthentification(this.serviceContexteAuthentification,
this.repositoryAuthentification)
 224 |   public serviceAuthentification = new ServiceAuthentification(
 225 |     this.serviceSessionAuthentification,
 226 |     this.serviceContexteAuthentification,
 227 |     this.serviceTotpSuperAdminAuthentification,
 228 |     this.serviceAutorisationAuthentification,
 229 |     this.serviceAuditAuthentification,
 230 |     this.serviceImpersonationAuthentification
 231 |   )
 232 |   public controleurAuthContext = new ControleurAuthContext(this.serviceAuthentification,
```

```
this.validateurAuthentification)
 233 |   public daoClientMemoire = new DaoClientMemoire()
 234 |   public daoClientPrisma = new DaoClientPrisma(this.prisma)
 235 |   public daoClient = this.utiliseMemoire ? this.daoClientMemoire : this.daoClientPrisma
 236 |   public daoDocumentMemoire = new DaoDocumentMemoire()
 237 |   public daoDocumentPrisma = new DaoDocumentPrisma(this.prisma)
 238 |   public daoDocument = this.utiliseMemoire ? this.daoDocumentMemoire :
this.daoDocumentPrisma
 239 |   public daoTransactionPaiementMemoire = new DaoTransactionPaiementMemoire()
 240 |   public daoTransactionPaiementPrisma = new DaoTransactionPaiementPrisma(this.prisma)
 241 |   public daoTransactionPaiement =
 242 |     this.utiliseMemoire ? this.daoTransactionPaiementMemoire :
this.daoTransactionPaiementPrisma
 243 |   public daoPaiementCautionMemoire = new DaoPaiementCautionMemoire()
 244 |   public daoPaiementCautionPrisma = new DaoPaiementCautionPrisma(this.prisma)
 245 |   public daoPaiementCaution =
 246 |     this.utiliseMemoire ? this.daoPaiementCautionMemoire : this.daoPaiementCautionPrisma
 247 |   public daoItemTravailMemoire = new DaoItemTravailMemoire()
 248 |   public daoItemTravailPrisma = new DaoItemTravailPrisma(this.prisma)
 249 |   public daoItemTravail = this.utiliseMemoire ? this.daoItemTravailMemoire :
this.daoItemTravailPrisma
 250 |   public daoExecutionImportMemoire = new DaoExecutionImportMemoire()
 251 |   public daoExecutionImportPrisma = new DaoExecutionImportPrisma(this.prisma)
 252 |   public daoExecutionImport =
 253 |     this.utiliseMemoire ? this.daoExecutionImportMemoire : this.daoExecutionImportPrisma
 254 |   public daoNotificationMemoire = new DaoNotificationMemoire()
 255 |   public daoNotificationPrisma = new DaoNotificationPrisma(this.prisma)
 256 |   public daoNotification = this.utiliseMemoire ? this.daoNotificationMemoire :
this.daoNotificationPrisma
 257 |   public daoIpBloqueeMemoire = new DaoIpBloqueeMemoire()
 258 |   public daoIpBloqueePrisma = new DaoIpBloqueePrisma(this.prisma)
 259 |   public daoIpBloquee = this.utiliseMemoire ? this.daoIpBloqueeMemoire :
this.daoIpBloqueePrisma
 260 |   public daoJournalAuditMemoire = new DaoJournalAuditMemoire()
```

BE-02 - Authentification Prisma avec normalisation telephone / email

La recherche d utilisateur accepte maintenant le telephone avec ou sans prefixe +221, ce qui stabilise l experience de connexion apres approbation d un admin.

Source : next-backend/src/infrastructure/dao/prisma/authentification/DaoAuthentificationPrisma.ts:16-120

```
  16 | export class DaoAuthentificationPrisma implements InterfaceDaoAuthentification {
  17 |   constructor(private readonly prisma: PrismaClient) {}
```

```
18 |
19 |   public async rechercherUtilisateurParTelephoneOuEmail(
20 |     identifiant: string
21 |   ): Promise<DonneesUtilisateurAuthentification | null> {
22 |     const identifiantNormalise = String(identifiant || '').trim()
23 |     const identifiantMinuscule = identifiantNormalise.toLowerCase()
24 |     const telephonesPossibles = this.construireTelephonesPossibles(identifiantNormalise)
25 |     const utilisateur = await this.prisma.utilisateur.findFirst({
26 |       where: {
27 |         OR: [
28 |           ...(telephonesPossibles.length > 0
29 |             ? [
30 |                 {
31 |                   telephone: {
32 |                     in: telephonesPossibles,
33 |                   },
34 |                 },
35 |               ]
36 |             : []),
37 |           {
38 |             email: {
39 |               equals: identifiantMinuscule,
40 |               mode: 'insensitive',
41 |             },
42 |           },
43 |         ],
44 |       },
45 |       include: {
46 |         permissions: {
47 |           where: { autorise: true },
48 |         },
49 |       },
50 |     })
51 |
52 |     if (!utilisateur) return null
53 |     return this.mapperUtilisateur(utilisateur)
54 |   }
55 |
56 |   private construireTelephonesPossibles(valeur: string): string[] {
57 |     const brut = String(valeur || '').trim()
58 |     if (!brut) return []
59 |
60 |     const compact = brut.replace(/[\s-]+/g, '')
61 |     const digits = compact.replace(/\D/g, '')
62 |     const candidats = new Set<string>()
63 |
64 |     if (compact) candidats.add(compact)
65 |     if (digits) candidats.add(digits)
66 |
67 |     if (/^7\d{8}$/.test(digits)) {
```

```
 68 |       candidats.add(`+221${digits}`)
 69 |       candidats.add(`221${digits}`)
 70 |     }
 71 |
 72 |     if (/^2217\d{8}$/.test(digits)) {
 73 |       candidats.add(`+${digits}`)
 74 |       candidats.add(digits.slice(3))
 75 |     }
 76 |
 77 |     return Array.from(candidats)
 78 |   }
 79 |
 80 |   public async rechercherUtilisateurParId(
 81 |     utilisateurId: string
 82 |   ): Promise<DonneesUtilisateurAuthentification | null> {
 83 |     const id = String(utilisateurId || '').trim()
 84 |     if (!id) return null
 85 |
 86 |     const utilisateur = await this.prisma.utilisateur.findUnique({
 87 |       where: { id },
 88 |       include: {
 89 |         permissions: {
 90 |           where: { autorise: true },
 91 |         },
 92 |       },
 93 |     })
 94 |
 95 |     if (!utilisateur) return null
 96 |     return this.mapperUtilisateur(utilisateur)
 97 |   }
 98 |
 99 |   public async creerSessionEtJetonRefresh(
100 |     entree: EntreeCreationSessionAuthentification
101 |   ): Promise<void> {
102 |     await this.prisma.$transaction(async (tx: Prisma.TransactionClient) => {
103 |       await tx.sessionAuthentification.create({
104 |         data: {
105 |           id: entree.sessionId,
106 |           utilisateurId: entree.utilisateurId,
107 |           jetonAccesJti: entree.jetonAccesJti,
108 |           jetonAccesExpireLe: entree.jetonAccesExpireLe,
109 |           csrfToken: entree.csrfToken,
110 |           adresseIp: entree.adresseIp,
111 |           agentUtilisateur: entree.agentUtilisateur,
112 |           expireLe: entree.expireLeSession,
113 |         },
114 |       })
115 |
116 |       await tx.jetonRefresh.create({
117 |         data: {
```

```
 118 |           sessionId: entree.sessionId,
 119 |           hachageToken: entree.hachageJetonRefresh,
 120 |           expireLe: entree.expireLeRefresh,
```

BE-03 - Route authContext

Cette route expose le contexte d authentification courant, adapte le format pour le frontend et nettoie l impersonation si le role ne le permet plus.

Source : next-backend/app/api/authContext/route.ts:1-80

```
   1 | import { NextRequest } from 'next/server'
   2 | import { conteneurDependances } from '@/src/coeur/conteneur/ConteneurDependances'
   3 | import { executerAvecGestionErreurs } from
'@/src/infrastructure/http/executerAvecGestionErreurs'
   4 | import { adapterUtilisateurAuthentifieFrontend } from
'@/src/infrastructure/http/adapterUtilisateurAuthentifieFrontend'
   5 |
   6 | /**
   7 |  * @swagger
   8 |  * /api/authContext:
   9 |  *   get:
  10 |  *     summary: Recupere le contexte d authentification courant
  11 |  *     tags:
  12 |  *       - Authentification
  13 |  *     security:
  14 |  *       - accessTokenCookie: []
  15 |  *       - bearerAuth: []
  16 |  *     responses:
  17 |  *       200:
  18 |  *         description: Contexte courant
  19 |  *       401:
  20 |  *         description: Non authentifie
  21 |  *       403:
  22 |  *         description: Origine non autorisee (CORS strict)
  23 |  */
  24 | export const GET = executerAvecGestionErreurs(
  25 |   conteneurDependances.reponseHttp,
  26 |   async (requete: NextRequest) => {
  27 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  28 |     const resultat = await
conteneurDependances.controleurAuthContext.contexte(jetonAcces)
  29 |     const role = String(resultat.user?.role || '').toUpperCase()
  30 |     const impersonation =
  31 |       role === 'SUPER_ADMIN'
```

```
  32 |         ? conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  33 |         : null
  34 |
  35 |     const reponse = conteneurDependances.reponseHttp.succes({
  36 |       ...resultat,
  37 |       user: adapterUtilisateurAuthentifieFrontend(resultat.user),
  38 |       impersonation,
  39 |     })
  40 |
  41 |     if (role !== 'SUPER_ADMIN') {
  42 |
conteneurDependances.serviceCookiesAuthentification.nettoyerCookieImpersonation(reponse)
  43 |     }
  44 |     return reponse
  45 |   }
  46 | )
  47 |
```

BE-04 - Route clients

La route clients montre la logique App Router, la securite CSRF, l extraction du jeton, l impersonation et le passage par le controleur metier.

Source : next-backend/app/api/clients/route.ts:1-80

```
   1 | import { NextRequest } from 'next/server'
   2 | import { conteneurDependances } from '@/src/coeur/conteneur/ConteneurDependances'
   3 | import { executerAvecGestionErreurs } from
'@/src/infrastructure/http/executerAvecGestionErreurs'
   4 | import { appliquerEntetesAnnulation } from
'@/src/infrastructure/http/appliquerEntetesAnnulation'
   5 | import { executerMutationIdempotenteSiDemandee } from
'@/src/infrastructure/http/executerMutationIdempotente'
   6 |
   7 | export const GET = executerAvecGestionErreurs(
   8 |   conteneurDependances.reponseHttp,
   9 |   async (requete: NextRequest) => {
  10 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  11 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  12 |     const donnees = await
conteneurDependances.controleurAdministrationAdmin.listerClients(
  13 |       jetonAcces,
  14 |       impersonation,
  15 |       new URL(requete.url)
```

```
  16 |     )
  17 |     return conteneurDependances.reponseHttp.succes(donnees)
  18 |   }
  19 | )
  20 |
  21 | export const POST = executerAvecGestionErreurs(
  22 |   conteneurDependances.reponseHttp,
  23 |   async (requete: NextRequest) => {
  24 |     conteneurDependances.adaptateurRequeteSecurite.exigerCsrf(requete)
  25 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  26 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  27 |     const corps = (await requete.json().catch(() => ({}))) as Record<string, unknown>
  28 |
  29 |     return executerMutationIdempotenteSiDemandee({
  30 |       prisma: conteneurDependances.prisma,
  31 |       requete,
  32 |       jetonAcces,
  33 |       impersonation,
  34 |       corps,
  35 |       serviceAuthentification: conteneurDependances.serviceAuthentification,
  36 |       executerMutation: async () => {
  37 |         const resultat = await
conteneurDependances.controleurAdministrationAdmin.creerClient(
  38 |           jetonAcces,
  39 |           impersonation,
  40 |           corps
  41 |         )
  42 |         const reponse = conteneurDependances.reponseHttp.succes(resultat.donnees)
  43 |         return appliquerEntetesAnnulation(reponse, resultat.annulation)
  44 |       },
  45 |     })
  46 |   }
  47 | )
  48 |
```

BE-05 - Route import runs

Le journal d import est conserve cote backend afin de rejouer les erreurs, d afficher les pages succes / erreurs et d assurer la traçabilite.

Source : next-backend/app/api/import_runs/route.ts:1-80

```
   1 | import { NextRequest } from 'next/server'
   2 | import { conteneurDependances } from '@/src/coeur/conteneur/ConteneurDependances'
```

```
   3 | import { executerAvecGestionErreurs } from
'@/src/infrastructure/http/executerAvecGestionErreurs'
   4 | import { appliquerEntetesAnnulation } from
'@/src/infrastructure/http/appliquerEntetesAnnulation'
   5 | import { executerMutationIdempotenteSiDemandee } from
'@/src/infrastructure/http/executerMutationIdempotente'
   6 |
   7 | export const GET = executerAvecGestionErreurs(
   8 |   conteneurDependances.reponseHttp,
   9 |   async (requete: NextRequest) => {
  10 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  11 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  12 |     const donnees = await
conteneurDependances.controleurAdministrationAdmin.listerImports(
  13 |       jetonAcces,
  14 |       impersonation
  15 |     )
  16 |     return conteneurDependances.reponseHttp.succes(donnees)
  17 |   }
  18 | )
  19 |
  20 | export const POST = executerAvecGestionErreurs(
  21 |   conteneurDependances.reponseHttp,
  22 |   async (requete: NextRequest) => {
  23 |     conteneurDependances.adaptateurRequeteSecurite.exigerCsrf(requete)
  24 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  25 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  26 |     const corps = (await requete.json().catch(() => ({}))) as Record<string, unknown>
  27 |
  28 |     return executerMutationIdempotenteSiDemandee({
  29 |       prisma: conteneurDependances.prisma,
  30 |       requete,
  31 |       jetonAcces,
  32 |       impersonation,
  33 |       corps,
  34 |       serviceAuthentification: conteneurDependances.serviceAuthentification,
  35 |       executerMutation: async () => {
  36 |         const resultat = await
conteneurDependances.controleurAdministrationAdmin.creerImport(
  37 |           jetonAcces,
  38 |           impersonation,
  39 |           corps
  40 |         )
  41 |         const reponse = conteneurDependances.reponseHttp.succes(resultat.donnees)
  42 |         return appliquerEntetesAnnulation(reponse, resultat.annulation)
  43 |       },
```

```
  44 |     })
  45 |   }
  46 | )
  47 |
```

BE-06 - Route paiements abonnement administrateur

Cette route controle les paiements d abonnement et declenche l evenement de notification Super Admin uniquement si le paiement est reelement finalise.

Source : next-backend/app/api/admin_payments/route.ts:1-120

```
   1 | import { NextRequest } from 'next/server'
   2 | import { conteneurDependances } from '@/src/coeur/conteneur/ConteneurDependances'
   3 | import { executerAvecGestionErreurs } from
'@/src/infrastructure/http/executerAvecGestionErreurs'
   4 | import { appliquerEntetesAnnulation } from
'@/src/infrastructure/http/appliquerEntetesAnnulation'
   5 | import { executerMutationIdempotenteSiDemandee } from
'@/src/infrastructure/http/executerMutationIdempotente'
   6 | import {
   7 |   estPaiementAbonnementFinalise,
   8 |   publierEvenementPaiementAbonnementAdminSuperAdmin,
   9 | } from '@/app/api/admin_payments/notificationPaiementsAdmin'
  10 |
  11 | export const GET = executerAvecGestionErreurs(
  12 |   conteneurDependances.reponseHttp,
  13 |   async (requete: NextRequest) => {
  14 |     const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  15 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  16 |     const donnees = await
conteneurDependances.controleurAdministrationAdmin.listerPaiementsAdmin(
  17 |       jetonAcces,
  18 |       impersonation
  19 |     )
  20 |     return conteneurDependances.reponseHttp.succes(donnees)
  21 |   }
  22 | )
  23 |
  24 | export const POST = executerAvecGestionErreurs(
  25 |   conteneurDependances.reponseHttp,
  26 |   async (requete: NextRequest) => {
  27 |     conteneurDependances.adaptateurRequeteSecurite.exigerCsrf(requete)
  28 |     const jetonAcces =
```

```
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  29 |     const impersonation =
conteneurDependances.adaptateurRequeteSecurite.lireImpersonation(requete)
  30 |     const corps = (await requete.json().catch(() => ({}))) as Record<string, unknown>
  31 |
  32 |     return executerMutationIdempotenteSiDemandee({
  33 |       prisma: conteneurDependances.prisma,
  34 |       requete,
  35 |       jetonAcces,
  36 |       impersonation,
  37 |       corps,
  38 |       serviceAuthentification: conteneurDependances.serviceAuthentification,
  39 |       executerMutation: async () => {
  40 |         const resultat = await
conteneurDependances.controleurAdministrationAdmin.creerPaiementAdmin(
  41 |           jetonAcces,
  42 |           impersonation,
  43 |           corps
  44 |         )
  45 |         if (estPaiementAbonnementFinalise(resultat.donnees as Record<string, unknown>))
{
  46 |           try {
  47 |             await publierEvenementPaiementAbonnementAdminSuperAdmin(
  48 |               resultat.donnees as Record<string, unknown>
  49 |             )
  50 |           } catch {
  51 |             // Notification best-effort.
  52 |           }
  53 |         }
  54 |         const reponse = conteneurDependances.reponseHttp.succes(resultat.donnees)
  55 |         return appliquerEntetesAnnulation(reponse, resultat.annulation)
  56 |       },
  57 |     })
  58 |   }
  59 | )
  60 |
```

BE-07 - Service de notification Brevo

Le service Brevo implemente une interface de notification, formate des templates HTML riches et envoie les emails transactionnels a chaque role concerne.

Source : next-backend/src/infrastructure/alertes/ServiceEmailBrevo.ts:1-220

```
   1 | import type {
   2 |   InterfaceNotification,
```

```
  3 |   TypeDestinataireNotification,
  4 |   TypeEntreeNotification,
  5 | } from '@/src/coeur/interfaces/InterfaceNotification'
  6 |
  7 | type TypeDestinataireEmailBrevo = {
  8 |   email: string
  9 |   name?: string
 10 | }
 11 |
 12 | type TypeEntreeEnvoiEmailBrevo = {
 13 |   destinataires: TypeDestinataireEmailBrevo[]
 14 |   sujet: string
 15 |   contenuTexte: string
 16 |   contenuHtml?: string
 17 |   tags?: string[]
 18 | }
 19 |
 20 | type TypeOptionsServiceEmailBrevo = {
 21 |   cleApi: string
 22 |   expediteurEmail: string
 23 |   expediteurNom?: string
 24 |   destinatairesParDefaut?: string[]
 25 | }
 26 |
 27 | type TypeLigneSectionNotification = {
 28 |   label: string
 29 |   valeur: string
 30 | }
 31 |
 32 | type TypeSectionNotification = {
 33 |   titre: string
 34 |   lignes: TypeLigneSectionNotification[]
 35 | }
 36 |
 37 | type TypeContenuNotification = {
 38 |   contenuTexte: string
 39 |   contenuHtml: string
 40 | }
 41 |
 42 | export class ServiceEmailBrevo implements InterfaceNotification {
 43 |   private static readonly URL_BREVO = 'https://api.brevo.com/v3/smtp/email'
 44 |
 45 |   constructor(private readonly options: TypeOptionsServiceEmailBrevo) {}
 46 |
 47 |   public estConfigure(): boolean {
 48 |     return Boolean(this.options.cleApi && this.options.expediteurEmail)
 49 |   }
 50 |
 51 |   public async notifier(entree: TypeEntreeNotification): Promise<boolean> {
 52 |     const details = this.versObjet(entree.details) || {}
```

```
  53 |     const destinataires = (entree.destinataires || [])
  54 |       .map((item) => this.mapperDestinataireNotification(item))
  55 |       .filter((item): item is TypeDestinataireEmailBrevo => Boolean(item))
  56 |     if (!destinataires.length) return false
  57 |
  58 |     const contenu = this.construireContenuParEvenement(entree, details)
  59 |     return this.envoyer({
  60 |       destinataires,
  61 |       sujet: this.normaliserTexte(entree.sujet) || 'Notification KYA',
  62 |       contenuTexte: contenu.contenuTexte,
  63 |       contenuHtml: contenu.contenuHtml,
  64 |       tags: entree.tags || [],
  65 |     })
  66 |   }
  67 |
  68 |   public async envoyerDemandeAdminCreee(donnees: Record<string, unknown>):
Promise<boolean> {
  69 |     const destinataires = this.extraireDestinataires(donnees)
  70 |     if (!destinataires.length) return false
  71 |     return this.notifier({
  72 |       evenement: 'ADMIN_REQUEST_CREATED',
  73 |       sujet: 'Nouvelle demande administrateur a valider',
  74 |       message: 'Une nouvelle demande administrateur a ete soumise.',
  75 |       details: donnees,
  76 |       tags: ['kya', 'admin-request', 'created'],
  77 |       destinataires: destinataires.map((item) => ({
  78 |         email: item.email,
  79 |         nom: item.name,
  80 |       })),
  81 |     })
  82 |   }
  83 |
  84 |   public async envoyerDemandeAdminApprouvee(donnees: Record<string, unknown>):
Promise<boolean> {
  85 |     const destinataires = this.extraireDestinataires(donnees)
  86 |     if (!destinataires.length) return false
  87 |     return this.notifier({
  88 |       evenement: 'ADMIN_REQUEST_APPROVED',
  89 |       sujet: 'Demande administrateur approuvee',
  90 |       message: 'Votre demande administrateur a ete approuvee.',
  91 |       details: donnees,
  92 |       tags: ['kya', 'admin-request', 'approved'],
  93 |       destinataires: destinataires.map((item) => ({
  94 |         email: item.email,
  95 |         nom: item.name,
  96 |       })),
  97 |     })
  98 |   }
  99 |
 100 |   public extraireDestinataires(donnees: Record<string, unknown>):
```

```
TypeDestinataireEmailBrevo[] {
 101 |     const uniques = new Map<string, TypeDestinataireEmailBrevo>()
 102 |
 103 |     for (const brut of this.options.destinatairesParDefaut || []) {
 104 |       const email = this.normaliserEmail(brut)
 105 |       if (!email) continue
 106 |       uniques.set(email.toLowerCase(), { email })
 107 |     }
 108 |
 109 |     const details = this.versObjet(donnees.details)
 110 |     const emailDemande = this.normaliserEmail(donnees.email) ||
this.normaliserEmail(details?.email)
 111 |     const nomDemande = this.normaliserTexte(donnees.name) ||
this.normaliserTexte(details?.name)
 112 |     if (emailDemande) {
 113 |       const cle = emailDemande.toLowerCase()
 114 |       uniques.set(cle, {
 115 |         email: emailDemande,
 116 |         ...(nomDemande ? { name: nomDemande } : {}),
 117 |       })
 118 |     }
 119 |
 120 |     return Array.from(uniques.values())
 121 |   }
 122 |
 123 |   public async envoyer(entree: TypeEntreeEnvoiEmailBrevo): Promise<boolean> {
 124 |     if (!this.estConfigure()) return false
 125 |
 126 |     const destinataires = (entree.destinataires || [])
 127 |       .map((item) => ({
 128 |         email: this.normaliserEmail(item.email),
 129 |         name: this.normaliserTexte(item.name),
 130 |       }))
 131 |       .filter((item): item is { email: string | null; name: string } =>
Boolean(item.email))
 132 |       .map((item) => ({
 133 |         email: item.email as string,
 134 |         ...(item.name ? { name: item.name } : {}),
 135 |       }))
 136 |     if (!destinataires.length) return false
 137 |
 138 |     const charge = {
 139 |       sender: {
 140 |         email: this.options.expediteurEmail,
 141 |         name: this.options.expediteurNom || 'Keur Ya Aicha',
 142 |       },
 143 |       to: destinataires,
 144 |       subject: entree.sujet,
 145 |       textContent: entree.contenuTexte,
 146 |       htmlContent: entree.contenuHtml || undefined,
```

```
147 |       tags: entree.tags || [],
148 |     }
149 |
150 |     try {
151 |       const reponse = await fetch(ServiceEmailBrevo.URL_BREVO, {
152 |         method: 'POST',
153 |         headers: {
154 |           Accept: 'application/json',
155 |           'Content-Type': 'application/json',
156 |           'api-key': this.options.cleApi,
157 |         },
158 |         body: JSON.stringify(charge),
159 |       })
160 |       if (!reponse.ok) {
161 |         const details = await reponse.text().catch(() => '')
162 |         console.error('[Brevo] Envoi email refuse', {
163 |           statut: reponse.status,
164 |           details: details.slice(0, 500),
165 |         })
166 |         return false
167 |       }
168 |       return true
169 |     } catch {
170 |       console.error('[Brevo] Echec technique pendant l envoi email')
171 |       return false
172 |     }
173 |   }
174 |
175 |   private construireContenuParEvenement(
176 |     entree: TypeEntreeNotification,
177 |     details: Record<string, unknown>
178 |   ): TypeContenuNotification {
179 |     const evenement = this.normaliserTexte(entree.evenement).toUpperCase()
180 |
181 |     if (evenement === 'CLIENT_PAYMENT_OVERDUE') {
182 |       return this.construireContenuClientPaiementEnRetard(entree, details)
183 |     }
184 |     if (evenement === 'ADMIN_SUBSCRIPTION_PAYMENT_RECORDED') {
185 |       return this.construireContenuPaiementAbonnementAdmin(entree, details)
186 |     }
187 |     if (evenement === 'ADMIN_REQUEST_CREATED') {
188 |       return this.construireContenuDemandeAdminCreee(entree, details)
189 |     }
190 |     if (evenement === 'ADMIN_REQUEST_APPROVED') {
191 |       return this.construireContenuDemandeAdminApprouvee(entree, details)
192 |     }
193 |
194 |     return this.construireContenuGenerique(entree, details)
195 |   }
196 |
```

```
 197 |   private construireContenuClientPaiementEnRetard(
 198 |     entree: TypeEntreeNotification,
 199 |     details: Record<string, unknown>
 200 |   ): TypeContenuNotification {
 201 |     const appName = this.resoudreAppName(details)
 202 |     const logoUrl = this.normaliserLogoPublique(details.logoUrl)
 203 |     const sections: TypeSectionNotification[] = [
 204 |       {
 205 |         titre: 'Client',
 206 |         lignes: [
 207 |           { label: 'Nom', valeur: this.lireTexte(details, ['clientName', 'name']) || '-'
},
 208 |           { label: 'Email', valeur: this.lireTexte(details, ['clientEmail', 'email']) ||
'-' },
 209 |           { label: 'Telephone', valeur: this.lireTexte(details, ['clientPhone',
'phone']) || '-' },
 210 |         ],
 211 |       },
 212 |       {
 213 |         titre: 'Paiement en retard',
 214 |         lignes: [
 215 |           { label: 'Bien', valeur: this.lireTexte(details, ['propertyName']) || '-' },
 216 |           { label: 'Date echeance', valeur: this.formaterDate(this.lireTexte(details,
['dueDate', 'dateEcheance'])) },
 217 |           { label: 'Jours retard', valeur: this.lireTexte(details, ['daysLate',
'joursRetard']) || '-' },
 218 |           { label: 'Montant du', valeur: this.formaterMontant(this.lireNombre(details,
['amountDue', 'montantDu'])) },
 219 |           { label: 'Montant paye', valeur: this.formaterMontant(this.lireNombre(details,
['amountPaid', 'montantPaye'])) },
 220 |           {
```

BE-08 - Relances clients impayes

La route de relance construit la liste des paiements en retard, calcule les montants restants, charge le contexte admin et envoie les notifications client / admin.

Source : next-backend/app/api/notifications/clients/impayes/route.ts:1-260

```
   1 | import { randomUUID } from 'crypto'
   2 | import { NextRequest } from 'next/server'
   3 | import { conteneurDependances } from '@/src/coeur/conteneur/ConteneurDependances'
   4 | import { executerAvecGestionErreurs } from
'@/src/infrastructure/http/executerAvecGestionErreurs'
   5 | import { lirePolitiquePlateforme } from '@/src/infrastructure/http/politiquePlateforme'
   6 | import { ErreurHttp } from '@/src/coeur/erreurs/ErreurHttp'
```

```
 7 | import { CODE_HTTP, ERRORS, t } from '@/src/messages'
 8 |
 9 | type TypeRelanceClient = {
10 |   notificationKey: string
11 |   paiementId: string
12 |   locationId: string
13 |   propertyName: string
14 |   dateEcheance: string
15 |   joursRetard: number
16 |   montantDu: number
17 |   montantPaye: number
18 |   montantRestant: number
19 |   adminId: string
20 |   clientId: string
21 |   clientNom: string
22 |   clientTelephone: string
23 |   clientEmail: string | null
24 | }
25 |
26 | type TypeContexteAdminRelance = {
27 |   adminId: string
28 |   adminName: string | null
29 |   adminEmail: string | null
30 |   companyName: string | null
31 |   logoUrl: string | null
32 |   appName: string
33 | }
34 |
35 | function convertirNombre(valeur: number | null | undefined): number {
36 |   if (typeof valeur !== 'number' || !Number.isFinite(valeur)) return 0
37 |   return Number(valeur.toFixed(2))
38 | }
39 |
40 | function normaliserTexte(valeur: unknown): string {
41 |   return String(valeur || '').trim()
42 | }
43 |
44 | function normaliserEmail(valeur: unknown): string | null {
45 |   const email = normaliserTexte(valeur)
46 |   if (!email) return null
47 |   const formatValide = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)
48 |   return formatValide ? email : null
49 | }
50 |
51 | function normaliserLogoPublique(valeur: unknown): string | null {
52 |   const url = normaliserTexte(valeur)
53 |   if (!url) return null
54 |   if (!/^https?:\/\//i.test(url)) return null
55 |   return url
56 | }
```

```
  57 |
  58 | function lireBooleenParam(valeur: string | null | undefined, defautValeur = false):
boolean {
  59 |   if (valeur == null) return defautValeur
  60 |   const normalise = String(valeur).trim().toLowerCase()
  61 |   if (!normalise) return defautValeur
  62 |   return ['1', 'true', 'yes', 'oui'].includes(normalise)
  63 | }
  64 |
  65 | function lireEntierParam(
  66 |   valeur: string | null | undefined,
  67 |   defautValeur: number,
  68 |   min: number,
  69 |   max: number
  70 | ): number {
  71 |   const nombre = Number(valeur)
  72 |   if (!Number.isFinite(nombre)) return defautValeur
  73 |   const entier = Math.floor(nombre)
  74 |   if (entier < min) return min
  75 |   if (entier > max) return max
  76 |   return entier
  77 | }
  78 |
  79 | function autoriserParSecretCron(requete: NextRequest): boolean {
  80 |   const secretConfigure =
conteneurDependances.configurationSecurite.cleCronRapportHebdoSuperAdmin()
  81 |   if (!secretConfigure) return false
  82 |   const secretRecu = String(requete.headers.get('x-cron-secret') || '').trim()
  83 |   if (!secretRecu) return false
  84 |   return secretRecu === secretConfigure
  85 | }
  86 |
  87 | async function exigerSuperAdminAvecSecondeAuth(requete: NextRequest): Promise<void> {
  88 |   const jetonAcces =
conteneurDependances.adaptateurRequeteSecurite.extraireJetonAcces(requete)
  89 |   const utilisateur = await
conteneurDependances.serviceAuthentification.exigerSecondeAuthSuperAdmin(jetonAcces)
  90 |   const role = String(utilisateur.role || '').toUpperCase()
  91 |   if (role !== 'SUPER_ADMIN') {
  92 |     throw new ErreurHttp(CODE_HTTP.INTERDIT, t(ERRORS.AUTH_ACCES_SUPER_ADMIN))
  93 |   }
  94 | }
  95 |
  96 | async function construireRelancesClients(now: Date, limite: number):
Promise<TypeRelanceClient[]> {
  97 |   const paiements = await conteneurDependances.prisma.paiementMensuel.findMany({
  98 |     where: { dateEcheance: { lt: now } },
  99 |     orderBy: { dateEcheance: 'asc' },
 100 |     take: limite,
 101 |     include: {
```

```
 102 |       location: {
 103 |         select: {
 104 |           id: true,
 105 |           nomBien: true,
 106 |           client: {
 107 |             select: {
 108 |               id: true,
 109 |               adminId: true,
 110 |               prenom: true,
 111 |               nom: true,
 112 |               telephone: true,
 113 |               email: true,
 114 |             },
 115 |           },
 116 |         },
 117 |       },
 118 |     },
 119 |   })
 120 |
 121 |   const jourReference = now.toISOString().slice(0, 10)
 122 |
 123 |   return paiements
 124 |     .filter((paiement) => convertirNombre(paiement.montantPaye) <
convertirNombre(paiement.montantDu))
 125 |     .map((paiement) => {
 126 |       const montantDu = convertirNombre(paiement.montantDu)
 127 |       const montantPaye = convertirNombre(paiement.montantPaye)
 128 |       const montantRestant = convertirNombre(Math.max(0, montantDu - montantPaye))
 129 |       const client = paiement.location.client
 130 |       const clientNom = `${String(client.prenom || '').trim()} ${String(client.nom ||
'').trim()}`.trim()
 131 |       const joursRetard = Math.max(
 132 |         1,
 133 |         Math.floor((now.getTime() - paiement.dateEcheance.getTime()) / (24 * 60 * 60 *
1000))
 134 |       )
 135 |
 136 |       return {
 137 |         notificationKey: `OVERDUE_REMINDER:${paiement.id}:${jourReference}`,
 138 |         paiementId: paiement.id,
 139 |         locationId: paiement.location.id,
 140 |         propertyName: String(paiement.location.nomBien || '').trim() || '-',
 141 |         dateEcheance: paiement.dateEcheance.toISOString(),
 142 |         joursRetard,
 143 |         montantDu,
 144 |         montantPaye,
 145 |         montantRestant,
 146 |         adminId: client.adminId,
 147 |         clientId: client.id,
 148 |         clientNom: clientNom || client.telephone,
```

```
 149 |         clientTelephone: client.telephone,
 150 |         clientEmail: client.email || null,
 151 |       }
 152 |     })
 153 | }
 154 |
 155 | function construireResumeAdmin(relances: TypeRelanceClient[]): Array<Record<string,
unknown>> {
 156 |   const parAdmin = new Map<string, { total: number; montantRestantTotal: number;
clients: Set<string> }>()
 157 |
 158 |   for (const relance of relances) {
 159 |     const item = parAdmin.get(relance.adminId) || {
 160 |       total: 0,
 161 |       montantRestantTotal: 0,
 162 |       clients: new Set<string>(),
 163 |     }
 164 |     item.total += 1
 165 |     item.montantRestantTotal += relance.montantRestant
 166 |     item.clients.add(relance.clientId)
 167 |     parAdmin.set(relance.adminId, item)
 168 |   }
 169 |
 170 |   return Array.from(parAdmin.entries()).map(([adminId, item]) => ({
 171 |     adminId,
 172 |     totalRelances: item.total,
 173 |     totalClientsImpactes: item.clients.size,
 174 |     montantRestantTotal: convertirNombre(item.montantRestantTotal),
 175 |   }))
 176 | }
 177 |
 178 | async function chargerContextesAdminRelances(
 179 |   relances: TypeRelanceClient[]
 180 | ): Promise<Map<string, TypeContexteAdminRelance>> {
 181 |   const adminIds = Array.from(new Set(relances.map((item) =>
normaliserTexte(item.adminId)).filter(Boolean)))
 182 |   const contextes = new Map<string, TypeContexteAdminRelance>()
 183 |   if (!adminIds.length) return contextes
 184 |
 185 |   const [admins, politique] = await Promise.all([
 186 |     conteneurDependances.prisma.admin.findMany({
 187 |       where: { id: { in: adminIds } },
 188 |       select: { id: true, nom: true, email: true, entrepriseId: true },
 189 |     }),
 190 |     lirePolitiquePlateforme(conteneurDependances.prisma).catch(() => null),
 191 |   ])
 192 |
 193 |   const entrepriseIds = Array.from(
 194 |     new Set(
 195 |       admins
```

```
196 |         .map((item) => normaliserTexte(item.entrepriseId))
197 |         .filter(Boolean)
198 |     )
199 |   )
200 |   const clesBranding = adminIds.flatMap((adminId) => [
201 |     `admin_branding_app_name:${adminId}`,
202 |     `admin_branding_logo_url:${adminId}`,
203 |   ])
204 |
205 |   const [entreprises, branding] = await Promise.all([
206 |     entrepriseIds.length
207 |       ? conteneurDependances.prisma.entreprise.findMany({
208 |           where: { id: { in: entrepriseIds } },
209 |           select: { id: true, nom: true },
210 |         })
211 |       : Promise.resolve([] as Array<{ id: string; nom: string }>),
212 |     clesBranding.length
213 |       ? conteneurDependances.prisma.parametreAdmin.findMany({
214 |           where: {
215 |             adminId: { in: adminIds },
216 |             cle: { in: clesBranding },
217 |           },
218 |           select: { adminId: true, cle: true, valeur: true },
219 |         })
220 |       : Promise.resolve([] as Array<{ adminId: string; cle: string; valeur: string }>),
221 |   ])
222 |
223 |   const entrepriseParId = new Map(entreprises.map((item) => [item.id, item.nom]))
224 |   const brandingParAdmin = new Map<string, { appName?: string; logoUrl?: string }>()
225 |   for (const item of branding) {
226 |     const adminId = normaliserTexte(item.adminId)
227 |     if (!adminId) continue
228 |     const brut = brandingParAdmin.get(adminId) || {}
229 |     if (item.cle === `admin_branding_app_name:${adminId}`) {
230 |       brut.appName = normaliserTexte(item.valeur)
231 |     }
232 |     if (item.cle === `admin_branding_logo_url:${adminId}`) {
233 |       brut.logoUrl = normaliserLogoPublique(item.valeur) || undefined
234 |     }
235 |     brandingParAdmin.set(adminId, brut)
236 |   }
237 |
238 |   const appNameParDefaut = normaliserTexte(politique?.branding.appName) || 'Keur Ya
Aicha'
239 |   const logoParDefaut = normaliserLogoPublique(politique?.branding.logoUrl)
240 |
241 |   for (const adminId of adminIds) {
242 |     const admin = admins.find((item) => item.id === adminId)
243 |     const brandingAdmin = brandingParAdmin.get(adminId) || {}
244 |     const entrepriseId = normaliserTexte(admin?.entrepriseId) || null
```

```
 245 |     const companyName = entrepriseId ?
normaliserTexte(entrepriseParId.get(entrepriseId)) || null : null
 246 |
 247 |     contextes.set(adminId, {
 248 |       adminId,
 249 |       adminName: normaliserTexte(admin?.nom) || null,
 250 |       adminEmail: normaliserEmail(admin?.email),
 251 |       companyName,
 252 |       logoUrl: brandingAdmin.logoUrl || logoParDefaut,
 253 |       appName: normaliserTexte(brandingAdmin.appName) || appNameParDefaut,
 254 |     })
 255 |   }
 256 |
 257 |   return contextes
 258 | }
 259 |
 260 | async function declencherEmailsRelancesClients(
```

DB-01 - Schema Prisma et modele relationnel

Le schema Prisma structure les comptes, les sessions, les permissions, les clients, les locations, les paiements, les imports, les notifications et l audit.

Source : next-backend/prisma/schema.prisma:1-260

```
   1 | generator client {
   2 |   provider = "prisma-client-js"
   3 | }
   4 |
   5 | datasource db {
   6 |   provider = "postgresql"
   7 |   url      = env("DATABASE_URL")
   8 | }
   9 |
  10 | model ConfigurationSysteme {
  11 |   id            String                   @id @default(cuid())
  12 |   cle           String
  13 |   valeurTexte   String?
  14 |   valeurNombre  Float?
  15 |   valeurBooleen Boolean?
  16 |   typeValeur    TypeValeurConfiguration
  17 |   portee        TypePorteeConfiguration  @default(GLOBAL)
  18 |   adminId       String?
  19 |   scopeCle      String                   @default("GLOBAL")
  20 |   origine       TypeOrigineConfiguration @default(DEFAULT)
  21 |   verrouille    Boolean                  @default(false)
```

```
22 |   creeLe        DateTime                @default(now())
23 |   misAJourLe    DateTime                @updatedAt
24 |
25 |   @@unique([cle, scopeCle])
26 |   @@index([portee, adminId])
27 |   @@map("configuration_systeme")
28 | }
29 |
30 | model Utilisateur {
31 |   id                     String                  @id @default(cuid())
32 |   telephone              String                  @unique @map("nomUtilisateur")
33 |   email                  String                  @unique
34 |   motDePasseHache        String
35 |   role                   TypeRoleUtilisateur
36 |   statut                 TypeStatutUtilisateur   @default(ACTIF)
37 |   superAdminTotpActive   Boolean                 @default(false)
38 |   superAdminTotpSecret   String?
39 |   creeLe                 DateTime                @default(now())
40 |   misAJourLe             DateTime                @updatedAt
41 |   sessions               SessionAuthentification[]
42 |   tentatives             TentativeConnexion[]
43 |   audits                 JournalAudit[]
44 |   permissions            PermissionUtilisateur[]
45 |   profilAdmin            Admin?
46 |
47 |   @@index([telephone])
48 |   @@index([role, statut])
49 |   @@map("utilisateurs")
50 | }
51 |
52 | model Admin {
53 |   id                          String                  @id @default(cuid())
54 |   utilisateurId               String                  @unique
55 |   nomUtilisateur              String
56 |   nom                         String
57 |   email                       String
58 |   statut                      TypeStatutAdmin         @default(EN_ATTENTE)
59 |   entrepriseId                String?
60 |   modeAbonnement              TypeModeAbonnementAdmin @default(monthly)
61 |   montantMensuelAbonnement    Float                   @default(0)
62 |   montantAnnuelAbonnement     Float                   @default(0)
63 |   autoriserMontantPersonnalise Boolean               @default(false)
64 |   permissionTableauDeBord     Boolean                 @default(true)
65 |   permissionClients           Boolean                 @default(true)
66 |   permissionLocations         Boolean                 @default(true)
67 |   permissionPaiements         Boolean                 @default(true)
68 |   permissionDocuments         Boolean                 @default(true)
69 |   permissionParametres        Boolean                 @default(true)
70 |   permissionTravaux           Boolean                 @default(true)
71 |   permissionImports           Boolean                 @default(true)
```

```
  72 |   permissionNotifications      Boolean                @default(true)
  73 |   permissionExportPdf          Boolean                @default(true)
  74 |   creeLe                       DateTime               @default(now())
  75 |   misAJourLe                   DateTime               @updatedAt
  76 |   utilisateur                  Utilisateur            @relation(fields:
[utilisateurId], references: [id], onDelete: Cascade)
  77 |
  78 |   @@index([statut, creeLe])
  79 |   @@index([entrepriseId])
  80 |   @@map("admins")
  81 | }
  82 |
  83 | model DemandeAdmin {
  84 |   id            String          @id @default(cuid())
  85 |   nom           String
  86 |   email         String?
  87 |   telephone     String?
  88 |   nomEntreprise String?
  89 |   statut        TypeStatutAdmin @default(EN_ATTENTE)
  90 |   nomUtilisateur String?
  91 |   motDePasse    String?
  92 |   paye          Boolean         @default(false)
  93 |   payeLe        DateTime?
  94 |   creeLe        DateTime        @default(now())
  95 |   misAJourLe    DateTime        @updatedAt
  96 |
  97 |   @@index([statut, creeLe])
  98 |   @@index([nomEntreprise])
  99 |   @@map("demandes_admin")
 100 | }
 101 |
 102 | model Entreprise {
 103 |   id         String   @id @default(cuid())
 104 |   nom        String
 105 |   adminId    String?
 106 |   creeLe     DateTime @default(now())
 107 |   misAJourLe DateTime @updatedAt
 108 |
 109 |   @@index([adminId])
 110 |   @@map("entreprises")
 111 | }
 112 |
 113 | model SessionAuthentification {
 114 |   id                        String                  @id @default(cuid())
 115 |   utilisateurId             String
 116 |   jetonAccesJti             String                  @unique
 117 |   jetonAccesExpireLe        DateTime
 118 |   csrfToken                 String
 119 |   adresseIp                 String?
 120 |   agentUtilisateur          String?
```

```
 121 |   secondeAuthValideeLe      DateTime?
 122 |   expireLe                  DateTime
 123 |   revoqueeLe                DateTime?
 124 |   compromissionDetecteeLe   DateTime?
 125 |   creeLe                    DateTime                @default(now())
 126 |   misAJourLe                DateTime                @updatedAt
 127 |   utilisateur               Utilisateur             @relation(fields: [utilisateurId],
references: [id], onDelete: Cascade)
 128 |   jetonsRefresh             JetonRefresh[]
 129 |
 130 |   @@index([utilisateurId, expireLe])
 131 |   @@index([revoqueeLe, compromissionDetecteeLe])
 132 |   @@map("sessions_authentification")
 133 | }
 134 |
 135 | model JetonRefresh {
 136 |   id              String         @id @default(cuid())
 137 |   sessionId       String
 138 |   hachageToken    String         @unique
 139 |   expireLe        DateTime
 140 |   utiliseLe       DateTime?
 141 |   revoqueLe       DateTime?
 142 |   remplaceParId   String?
 143 |   creeLe          DateTime       @default(now())
 144 |   session         SessionAuthentification @relation(fields: [sessionId], references:
[id], onDelete: Cascade)
 145 |   remplacePar     JetonRefresh?  @relation("rotation_jetons_refresh", fields:
[remplaceParId], references: [id])
 146 |   jetonsRemplaces JetonRefresh[] @relation("rotation_jetons_refresh")
 147 |
 148 |   @@index([sessionId, expireLe])
 149 |   @@index([revoqueLe, utiliseLe])
 150 |   @@map("jetons_refresh")
 151 | }
 152 |
 153 | model TentativeConnexion {
 154 |   id             String                  @id @default(cuid())
 155 |   identifiant    String
 156 |   adresseIp      String
 157 |   succes         Boolean
 158 |   type           TypeTentativeConnexion
 159 |   creeLe         DateTime                @default(now())
 160 |   utilisateurId  String?
 161 |   utilisateur    Utilisateur?            @relation(fields: [utilisateurId], references:
[id], onDelete: SetNull)
 162 |
 163 |   @@index([identifiant, adresseIp, type, creeLe])
 164 |   @@map("tentatives_connexion")
 165 | }
 166 |
```

```
 167 | model JournalAudit {
 168 |   id              String       @id @default(cuid())
 169 |   utilisateurId   String?
 170 |   action          String
 171 |   statut          String
 172 |   details         String?
 173 |   adresseIp       String?
 174 |   agentUtilisateur String?
 175 |   creeLe          DateTime     @default(now())
 176 |   utilisateur     Utilisateur? @relation(fields: [utilisateurId], references: [id],
onDelete: SetNull)
 177 |
 178 |   @@index([action, statut, creeLe])
 179 |   @@index([utilisateurId, creeLe])
 180 |   @@map("journaux_audit")
 181 | }
 182 |
 183 | model PermissionUtilisateur {
 184 |   id             String       @id @default(cuid())
 185 |   utilisateurId  String
 186 |   code           String
 187 |   autorise       Boolean      @default(false)
 188 |   creeLe         DateTime     @default(now())
 189 |   misAJourLe     DateTime     @updatedAt
 190 |   utilisateur    Utilisateur  @relation(fields: [utilisateurId], references: [id],
onDelete: Cascade)
 191 |
 192 |   @@unique([utilisateurId, code])
 193 |   @@index([code, autorise])
 194 |   @@map("permissions_utilisateur")
 195 | }
 196 |
 197 | model Client {
 198 |   id         String     @id
 199 |   adminId    String
 200 |   prenom     String
 201 |   nom        String
 202 |   telephone  String
 203 |   cni        String
 204 |   email      String?
 205 |   statut     String     @default("active")
 206 |   creeLe     DateTime   @default(now())
 207 |   misAJourLe DateTime   @updatedAt
 208 |   locations  Location[]
 209 |
 210 |   @@index([adminId, creeLe])
 211 |   @@index([telephone])
 212 |   @@map("clients")
 213 | }
 214 |
```

```
 215 | model Location {
 216 |   id                  String            @id
 217 |   clientId            String
 218 |   typeBien            String
 219 |   nomBien             String
 220 |   loyerMensuel        Float
 221 |   dateDebut           DateTime
 222 |   cautionMontantTotal Float             @default(0)
 223 |   cautionMontantPaye  Float             @default(0)
 224 |   creeLe              DateTime          @default(now())
 225 |   misAJourLe          DateTime          @updatedAt
 226 |   client              Client            @relation(fields: [clientId], references: [id],
onDelete: Cascade)
 227 |   documents           Document[]
 228 |   paiementsMensuels   PaiementMensuel[]
 229 |   depots              PaiementCaution[]
 230 |
 231 |   @@index([clientId, creeLe])
 232 |   @@map("locations")
 233 | }
 234 |
 235 | model Document {
 236 |   id         String    @id
 237 |   locationId String?
 238 |   nom        String
 239 |   type       String
 240 |   url        String
 241 |   dateAjout  DateTime
 242 |   estSigne   Boolean   @default(false)
 243 |   creeLe     DateTime  @default(now())
 244 |   misAJourLe DateTime  @updatedAt
 245 |   location   Location? @relation(fields: [locationId], references: [id], onDelete:
Cascade)
 246 |
 247 |   @@index([locationId, creeLe])
 248 |   @@map("documents")
 249 | }
 250 |
 251 | model PaiementMensuel {
 252 |   id          String               @id
 253 |   locationId  String
 254 |   periodeDebut DateTime
 255 |   periodeFin  DateTime
 256 |   dateEcheance DateTime
 257 |   montantDu   Float
 258 |   montantPaye Float                @default(0)
 259 |   statut      String               @default("unpaid")
 260 |   creeLe      DateTime             @default(now())
```

Annexe F - Sources Mermaid utilisees

Use case global

```
# Aspect Fonctionnel - Analyse - Diagramme de Cas d Utilisation

## Diagramme global

```mermaid
flowchart LR
  SA["SUPER_ADMIN"] --> UC1["Gerer demandes admin"]
  SA --> UC2["Activer seconde authentification"]
  SA --> UC3["Impersoner un admin"]
  SA --> UC4["Bloquer/Debloquer IP"]
  SA --> UC5["Activer/Desactiver maintenance"]
  SA --> UC6["Valider paiement cash abonnement"]
  SA --> UC7["Consulter audit et supervision"]

  AD["ADMIN"] --> UC8["Se connecter"]
  AD --> UC9["Gerer clients"]
  AD --> UC10["Gerer locations et documents"]
  AD --> UC11["Payer abonnement"]
  AD --> UC12["Consulter statut abonnement"]
  AD --> UC13["Importer clients"]
  AD --> UC14["Executer rollback autorise"]

  OPS["OPS/SUPPORT"] --> UC15["Analyser incidents"]
  OPS --> UC16["Suivre imports en erreur"]
  OPS --> UC17["Consulter audit logs"]

  PSP["SYSTEME PAIEMENT"] --> UC18["Envoyer webhook signe"]
  UC18 --> UC19["Confirmer paiement admin"]
```

Use cases par acteur

SUPER_ADMIN

```mermaid
flowchart LR
  SA["SUPER_ADMIN"] --> S1["Se connecter"]
  SA --> S2["Second auth"]
  SA --> S3["Impersonation"]
  SA --> S4["Valider/Rejeter admin_request"]
  SA --> S5["Bloquer IP"]
  SA --> S6["Debloquer IP"]
  SA --> S7["Valider paiement cash"]
```
```

```
  SA --> S8["Maintenance policy"]
```

ADMIN

```mermaid
flowchart LR
  AD["ADMIN"] --> A1["Login"]
  AD --> A2["CRUD clients"]
  AD --> A3["CRUD rentals"]
  AD --> A4["Gerer documents"]
  AD --> A5["Payer abonnement mobile money"]
  AD --> A6["Importer clients"]
  AD --> A7["Undo personnels"]
```

OPS/SUPPORT

```mermaid
flowchart LR
  OPS["OPS/SUPPORT"] --> O1["Audit consultation"]
  OPS --> O2["Triage incidents login"]
  OPS --> O3["Suivi import_runs/import_errors"]
```

SYSTEME PAIEMENT

```mermaid
flowchart LR
  PSP["SYSTEME PAIEMENT"] --> P1["Webhook signe"]
  P1 --> P2["Verification signature"]
  P2 --> P3["Passage admin_payment en paid"]
```

Activite abonnement

```
# Aspect Fonctionnel - Analyse - Diagramme d Activite

## Activite transversale: abonnement admin

```mermaid
flowchart LR
  A["Debut"] --> B["Admin authentifie"]
  B --> C["Saisir paiement abonnement"]
  C --> D{"Methode valide ?"}
  D -->|Non| X1["Erreur validation"]
  D -->|Oui| E{"Mois requis respecte ?"}
```
```

```
  E -->|Non| X2["Blocage mois prioritaire"]
  E -->|Oui| F{"Doublon actif (admin,mois) ?"}
  F -->|Oui| X3["Rejet doublon"]
  F -->|Non| G{"cash ?"}
  G -->|Oui| H["Validation Super Admin"]
  G -->|Non| I["Initiation provider"]
  I --> J{"Webhook paid ?"}
  J -->|Oui| K["Statut paid + audit"]
  J -->|Non| L["Statut pending"]
  H --> K
  K --> M["Fin succes"]
  L --> M
```
```

## Architecture globale

```
Aspect Architectural - Diagramme d Architecture Globale

```mermaid
flowchart LR
  FE["Frontend React/Electron"] --> API["KYA API"]
  API --> SVC["Services Metier"]
  SVC --> REPO["Repositories"]
  REPO --> DB[(PostgreSQL)]

  API --> SEC["Middlewares\nAuth/RBAC/Ownership/Maintenance"]
  API --> EXT1["Provider Paiement"]
  API --> EXT2["Cloudinary"]

  API --> OBS["Logs/Audit/Metrics"]
  CI["Jenkins CI/CD"] --> API
```
```

## Composants backend

```
Aspect Architectural - Diagramme de Composants

```mermaid
flowchart LR
  subgraph API["API Layer"]
    R["Routes"] --> C["Controllers"]
    C --> V["Validation"]
    C --> D["DTO Mapper"]
    C --> S["Services"]
```
```
```

```
    S --> I["Repository Interfaces"]
    I --> P["PostgreSQL Repositories"]
    C --> M["Message Catalog"]
  end

  subgraph SEC["Security"]
    MW1["Auth Middleware"]
    MW2["Ownership Middleware"]
    MW3["Maintenance Middleware"]
    MW4["RateLimit/IP Block Middleware"]
  end

  C --> MW1
  C --> MW2
  C --> MW3
  C --> MW4
  P --> DB[(PostgreSQL)]
  S --> PSP["Payment Adapter"]
  S --> CLD["Cloudinary Adapter"]
```

Diagrammes de sequence

```
# Diagrammes de Sequence

## Login + second auth

```mermaid
sequenceDiagram
  participant User as User
  participant API as /authContext/login
  participant Sec as Security Logic
  participant Data as Store/DB

  User->>API: POST login(username,password)
  API->>Sec: verifyCredentials()
  Sec->>Data: findUserByLogin()
  alt invalid credentials
    Sec->>Data: appendAuditLog(FAILED_LOGIN)
    Sec->>Data: maybeBlockIp()
    API-->>User: 401 {error}
  else valid admin but pending/inactive
    API-->>User: 403 pending approval
  else valid
    Sec->>Data: setAuthContext(userId)
    API-->>User: 200 authContext + subscription flags
  end
```
```

```
  User->>API: POST /authContext/super-admin/second-auth
  API->>Sec: verifySuperAdminPassword()
  alt wrong password
    Sec->>Data: appendAuditLog(SUPER_ADMIN_SECOND_AUTH_FAILED)
    API-->>User: 401
  else success
    Sec->>Data: set superAdminSecondAuthAt(now)
    Sec->>Data: appendAuditLog(SUPER_ADMIN_SECOND_AUTH_SUCCESS)
    API-->>User: 200 {ok:true}
  end
```

Creation paiement admin + webhook

```mermaid
sequenceDiagram
  participant Admin as Admin
  participant API as /admin_payments
  participant Svc as Payment Service
  participant PSP as Provider
  participant DB as Store/DB

  Admin->>API: POST payment(amount,method,month)
  API->>Svc: validate role/method/amount/month
  Svc->>Svc: compute requiredMonth
  Svc->>DB: check duplicate active(admin,month)
  alt duplicate or wrong month order
    API-->>Admin: 409
  else mobile method
    Svc->>PSP: initiate payment
    PSP-->>Svc: pending/paid + reference
    Svc->>DB: insert admin_payment
    API-->>Admin: 201 {status pending|paid}
  else cash by super admin
    Svc->>DB: insert admin_payment(status=paid)
    API-->>Admin: 201
  end

  PSP->>API: POST /admin_payments/webhook/:provider
  API->>Svc: verify signature
  alt invalid signature
    API-->>PSP: 401
  else paid event
    Svc->>DB: locate payment
    Svc->>DB: mark paid + update admin paid flag
    API-->>PSP: 200 {ok:true}
  end
```

Import clients + erreurs

```mermaid
sequenceDiagram
  participant Admin as Admin
  participant API as /import_runs
  participant Svc as Import Service
  participant DB as Store/DB

  Admin->>API: POST import run(file metadata + parsed data)
  API->>Svc: resolve owner adminId
  Svc->>DB: create import_run
  loop each row
    Svc->>Svc: validate row
    alt invalid or duplicate
      Svc->>DB: append import_error
    else valid
      Svc->>DB: create/update client
    end
  end
  Svc->>DB: update run counters/status
  API-->>Admin: 201 import summary
```

Undo rollback

```mermaid
sequenceDiagram
  participant U as User
  participant API as /undo-actions/:id/rollback
  participant Undo as Undo Service
  participant DB as Store/DB

  U->>API: POST rollback(id)
  API->>Undo: verify auth + ownership
  Undo->>DB: load undo entry
  alt not found/forbidden/expired
    API-->>U: 404|403|410
  else valid
    Undo->>Undo: resolve rollback plan
    Undo->>DB: apply rollback + side effects
    Undo->>DB: remove undo entry
    Undo->>DB: append audit log
    API-->>U: 200 {ok:true}
  end
```